

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 1 – Introduction, Architecture Vision & Design Principles

Document Version: 1.0

Project: Smart Education Platform

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Technology Stack: Java 21, Spring Boot 3.x, React.js, PostgreSQL, Redis, Docker

1.1 Introduction

The **Smart Education Platform** is an AI-powered, enterprise-grade learning and placement ecosystem designed to bridge the gap between education and employment.

The platform provides learners with personalized learning experiences, coding practice, aptitude preparation, AI coaching, resume building, gamification, placement opportunities, and recruiter engagement through a unified, scalable, and secure architecture.

The architecture is designed to support millions of users while maintaining high performance, security, maintainability, and future scalability.

1.2 Vision

To build India's most intelligent AI-driven learning and placement ecosystem that enables learners to become industry-ready professionals and helps recruiters hire the right talent efficiently.

1.3 Mission

- Deliver world-class learning experiences.

- Improve student employability.
 - Automate placement activities.
 - Enable AI-powered career guidance.
 - Support enterprise-scale recruitment.
 - Build a highly scalable and secure platform.
-

1.4 Architecture Goals

The architecture is designed to achieve:

- Scalability
 - High Availability
 - Security
 - Performance
 - Maintainability
 - Extensibility
 - Fault Tolerance
 - Modular Development
 - Cloud Readiness
 - AI Integration
-

1.5 Business Objectives

The platform will provide:

Independent Learner Portal

- Learning Dashboard
- Brain Gym
- Aptitude Arena
- Coding Arena
- SQL Arena
- Debug Hero
- Resume Builder
- AI Coach
- Placement Portal
- Gamification

Recruiter Portal

- Company Management
- Job Posting
- Candidate Search
- Hiring Pipeline
- Interview Management
- Offer Management
- Recruitment Analytics

Super Admin Portal

- User Management
 - Course Management
 - Assessment Management
 - Recruiter Management
 - AI Management
 - Reports & Analytics
 - System Configuration
 - Audit & Monitoring
-

1.6 Architecture Principles

The Smart Education Platform follows these core principles:

1. Clean Architecture

- Separation of concerns
- Independent business logic
- Framework independence
- Testability

2. SOLID Principles

- Single Responsibility
- Open/Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

3. Domain-Driven Design (DDD)

Business modules are organized around core domains:

- Authentication
- Learner
- Assessment
- AI Coach
- Resume
- Placement
- Recruiter
- Gamification
- Notification
- Reports

4. API-First Design

All business functionality is exposed through secure REST APIs.

5. Security by Design

Security is integrated into every architectural layer.

1.7 Functional Scope

The platform includes the following major modules:

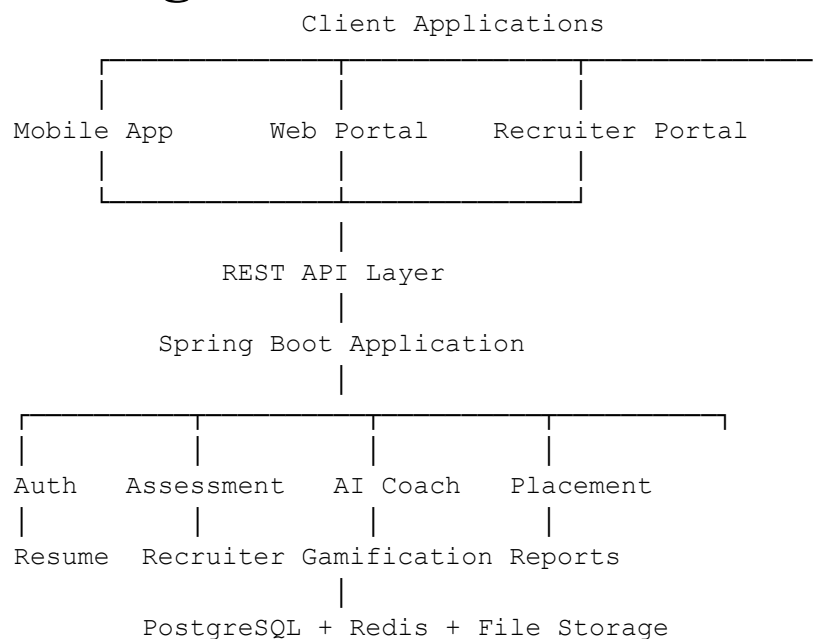
Module	Description
Authentication	Login, Registration, JWT Security
Learner	Profile & Career Management
Brain Gym	Logical Thinking Exercises
Aptitude Arena	Placement Aptitude Tests
Coding Arena	Coding Practice & Evaluation
SQL Arena	SQL Practice
Debug Hero	Debugging Challenges
AI Coach	AI Career & Learning Assistant
Resume Builder	ATS-Friendly Resume Creation
Placement Portal	Job Applications & Interviews
Recruiter Portal	Recruitment Management
Gamification	XP, Badges & Rewards
Notifications	Email, SMS & Push
Reports	Analytics & Dashboards

1.8 Non-Functional Requirements

The architecture is designed to meet the following quality attributes:

- High Performance
 - Horizontal Scalability
 - 99.9% Availability
 - Secure Authentication
 - Fault Tolerance
 - Observability
 - Data Integrity
 - Maintainability
 - Extensibility
 - Disaster Recovery Support
-

1.9 High-Level Architecture Overview



1.10 Quality Attributes

The architecture prioritizes:

- Reliability
 - Availability
 - Performance
 - Security
 - Modularity
 - Scalability
 - Observability
 - Recoverability
 - Flexibility
 - Maintainability
-

1.11 Design Principles

The platform follows these engineering standards:

- Layered Architecture
 - Dependency Injection
 - Stateless REST APIs
 - DTO-Based Communication
 - Repository Pattern
 - Service Layer Pattern
 - Event-Driven Processing
 - Asynchronous Tasks
 - Centralized Exception Handling
 - Structured Logging
-

1.12 Technology Vision

The platform is designed to support:

- AI-assisted learning
- Real-time analytics
- Cloud-native deployment
- Containerized infrastructure
- Horizontal scaling
- Future microservice migration
- Multi-tenant support (future)
- Multi-language support (future)

1.13 Assumptions

- All users authenticate using JWT-based security.
 - PostgreSQL is the primary relational database.
 - Redis is used for caching and session optimization.
 - Object storage is used for documents and media.
 - REST APIs serve all client applications.
 - Docker is used for deployment.
 - CI/CD pipelines automate delivery.
-

1.14 Architecture Success Metrics

The architecture will be considered successful if it achieves:

- API response time below 300 ms for common operations
 - High availability with minimal downtime
 - Support for thousands of concurrent users
 - Secure handling of user and company data
 - Modular codebase for independent feature development
 - Automated deployment and monitoring
 - Easy integration of future AI capabilities
-

1.15 Chapter Summary

This chapter introduces the **Software Architecture Document (SAD)** for the Smart Education Platform. It defines the overall vision, business goals, architectural principles, functional scope, non-functional requirements, and quality attributes that guide the design and implementation of the platform. It establishes the foundation for a secure, scalable, maintainable, and AI-ready enterprise application.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 2 – Technology Stack, Architecture Style & Technology Decision Matrix

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Technology Baseline: Java 21 + Spring Boot 3.x + React + PostgreSQL

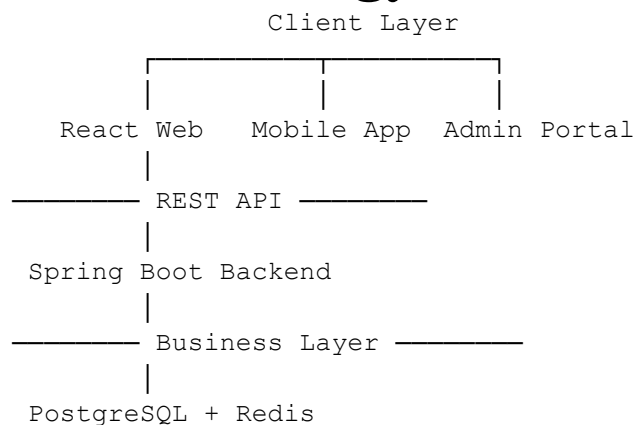
2.1 Introduction

Selecting the right technology stack is one of the most important architectural decisions for an enterprise application.

The Smart Education Platform is designed using **modern, open-source, cloud-ready technologies** that provide:

- High Performance
 - Enterprise Security
 - Scalability
 - Easy Maintenance
 - Large Community Support
 - AI Integration
 - Cloud Deployment
 - Future Microservice Migration
-

2.2 Technology Architecture Overview



2.3 Complete Technology Stack

Layer	Technology
Frontend	React.js 19
Mobile	React Native / Flutter (Future)
Backend	Java 21
Framework	Spring Boot 3.x
Security	Spring Security + JWT
ORM	Spring Data JPA + Hibernate
Database	PostgreSQL 16+
Cache	Redis
Build Tool	Maven
API	REST APIs
Documentation	OpenAPI (Swagger)
Containerization	Docker
Reverse Proxy	Nginx
CI/CD	GitHub Actions
Monitoring	Prometheus + Grafana
Logging	ELK Stack
Object Storage	MinIO / AWS S3
AI Integration	Spring AI

2.4 Frontend Technology

React.js

Reasons

- Component-based architecture

- Excellent ecosystem
- Fast rendering
- Large community
- Easy API integration
- Enterprise support

Libraries

- React Router
 - Axios
 - Redux Toolkit
 - React Query
 - Material UI
 - Tailwind CSS
 - React Hook Form
 - Chart.js
-

2.5 Backend Technology

Java 21

Reasons

- Long-Term Support (LTS)
 - High Performance
 - Enterprise Standard
 - Strong Type Safety
 - Virtual Threads Support
 - Large Ecosystem
-

Spring Boot 3.x

Modules

- Spring MVC
- Spring Security
- Spring Data JPA
- Spring Validation
- Spring Cache
- Spring Scheduler

- Spring Mail
- Spring AI

Reasons

- Rapid Development
 - Enterprise Ready
 - Production Stable
 - Large Community
 - Excellent Testing Support
-

2.6 Database Technology

PostgreSQL

Reasons

- ACID Compliance
 - Excellent Performance
 - JSONB Support
 - Full-Text Search
 - Materialized Views
 - Partitioning
 - Advanced Indexing
 - Strong Security
-

2.7 Cache Technology

Redis

Purpose

- Session Cache
- JWT Blacklist
- OTP Storage
- Dashboard Cache
- Leaderboards
- AI Response Cache
- Frequently Used Data

Benefits

- Extremely Fast
 - In-Memory Storage
 - Distributed Cache
 - TTL Support
-

2.8 Object Storage

Stores

- Resume PDFs
- Profile Photos
- Company Logos
- Documents
- Offer Letters
- Certificates
- Videos

Recommended

- MinIO (Self-hosted)
 - AWS S3 (Cloud)
-

2.9 AI Technology

Spring AI

Supported Features

- AI Coach
- Resume Review
- Mock Interview
- Career Guidance
- Coding Assistance
- SQL Assistance
- Learning Recommendations

Supported AI Providers

- OpenAI
 - Azure OpenAI
 - Google Gemini
 - Anthropic Claude
 - Local LLMs (Future)
-

2.10 Security Technology

Authentication

- JWT
- Refresh Token

Authorization

- Role-Based Access Control (RBAC)
- Permission-Based Access

Security Components

- Spring Security
 - BCrypt Password Hashing
 - HTTPS
 - CORS
 - CSRF Protection (where applicable)
 - Rate Limiting
-

2.11 API Technology

Architecture

REST APIs

Documentation

- OpenAPI 3
- Swagger UI

Response Format

```
{  
  "success": true,
```

```
"message": "Operation completed",  
"data": {},  
"timestamp": "2026-01-01T10:00:00Z"  
}
```

2.12 DevOps Technology

Component	Technology
-----------	------------

Version Control	Git
-----------------	-----

Repository	GitHub
------------	--------

Build	Maven
-------	-------

CI/CD	GitHub Actions
-------	----------------

Container	Docker
-----------	--------

Reverse Proxy	Nginx
---------------	-------

SSL	Let's Encrypt
-----	---------------

Monitoring	Prometheus
------------	------------

Dashboard	Grafana
-----------	---------

2.13 Logging Technology

Recommended Stack

- Logback
- ELK Stack
- Loki (Alternative)

Logs

- Application Logs
 - Security Logs
 - Audit Logs
 - API Logs
 - Error Logs
-

2.14 Monitoring Technology

Monitor

- CPU
- Memory
- Database
- Redis
- APIs
- JVM
- Docker Containers

Tools

- Prometheus
 - Grafana
-

2.15 Architecture Style

The platform follows an **Enterprise Modular Monolith** architecture.

Spring Boot

└─ Authentication

└─ Learner

└─ Assessment

└─ AI Coach

└─ Resume

└─ Placement

└─ Recruiter

└─ Notification

└─ Reports

└─ Shared

Benefits

- Faster development
- Easier deployment

- Simpler debugging
 - Strong module isolation
 - Microservice-ready structure
-

2.16 Technology Decision Matrix

Requirement	Selected Technology	Reason
Programming Language	Java 21	Enterprise standard, LTS
Backend Framework	Spring Boot 3.x	Mature ecosystem
Frontend	React.js	Rich UI ecosystem
Database	PostgreSQL	Advanced SQL features
Cache	Redis	High-speed in-memory cache
Object Storage	MinIO	S3-compatible, self-hosted
AI Integration	Spring AI	Multi-provider support
Build Tool	Maven	Industry standard
Containerization	Docker	Consistent deployments
CI/CD	GitHub Actions	Integrated automation

2.17 Version Compatibility

Technology	Version
Java	21 LTS
Spring Boot	3.x
Spring Security	6.x
Hibernate	6.x
PostgreSQL	16+
Redis	7+
React	19
Maven	3.9+
Docker	Latest Stable
Nginx	Latest Stable

2.18 Technology Selection Principles

Selection was based on:

- Enterprise Stability
 - Long-Term Support
 - Performance
 - Security
 - Scalability
 - Community Support
 - Cloud Readiness
 - AI Compatibility
 - Developer Productivity
-

2.19 Future Technology Roadmap

Future enhancements may include:

- Kubernetes
 - Apache Kafka
 - Elasticsearch
 - GraphQL APIs
 - gRPC
 - Multi-Tenant Architecture
 - Microservices Migration
 - Event-Driven Architecture
 - Serverless AI Services
 - Multi-Cloud Deployment
-

2.20 Chapter Summary

This chapter defines the complete **Technology Stack, Architecture Style, and Technology Decision Matrix** for the Smart Education Platform. The selected technologies provide a robust foundation for building a secure, scalable, cloud-ready, AI-powered enterprise application. The modular monolith approach enables rapid development today while keeping the architecture ready for future migration to microservices.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 3 – Enterprise Architecture

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

3.1 Introduction

The **Enterprise Architecture** defines the overall structural design of the Smart Education Platform. It establishes how different layers, modules, services, and infrastructure components interact while ensuring scalability, maintainability, security, and high performance.

The architecture follows **Clean Architecture**, **Domain-Driven Design (DDD)**, **SOLID Principles**, and **Layered Architecture** to create a modular, enterprise-grade application.

3.2 Architecture Goals

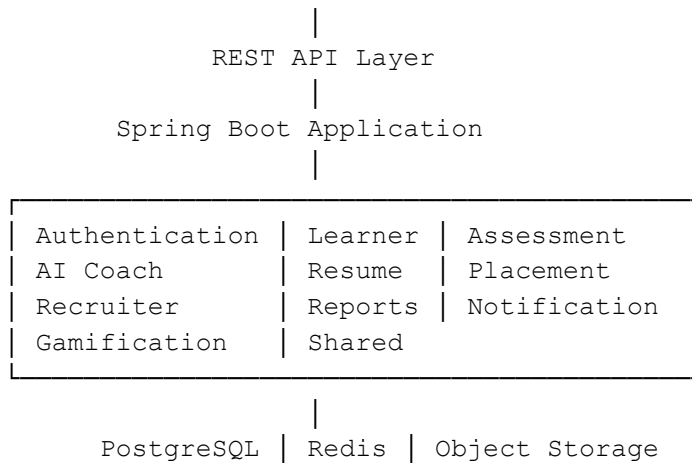
The enterprise architecture is designed to achieve:

- Modular Development
 - Loose Coupling
 - High Cohesion
 - Scalability
 - Security
 - Performance
 - Testability
 - Reusability
 - Cloud Readiness
 - Future Microservice Migration
-

3.3 Enterprise Architecture Overview

Client Layer

React Web		Mobile App		Recruiter Portal	
-----------	--	------------	--	------------------	--



3.4 Architecture Layers

The application is divided into multiple logical layers.

Presentation Layer

Responsibilities:

- React UI
- REST API Requests
- Input Validation
- Authentication Token Handling
- User Interaction

API Layer

Responsibilities:

- REST Controllers
- Request Validation
- Response Formatting
- Exception Handling
- API Documentation

Business Layer

Responsibilities:

- Business Rules
 - Workflow Management
 - Transaction Management
 - Module Coordination
 - Domain Services
-

Persistence Layer

Responsibilities:

- JPA Repositories
 - Entity Management
 - Database Operations
 - Query Optimization
 - Transaction Support
-

Infrastructure Layer

Responsibilities:

- Redis
 - File Storage
 - Email
 - SMS
 - AI Integration
 - External APIs
 - Scheduler
 - Monitoring
-

3.5 Clean Architecture

Presentation Layer

↓

Controller Layer

↓

Service Layer

↓

Repository Layer

↓

Database

Dependency Rule

Dependencies always point inward.

- UI depends on API
- API depends on Services
- Services depend on Repositories
- Repositories depend on Database

Business logic never depends directly on UI frameworks.

3.6 Domain Modules

The application is organized into business domains.

Module	Responsibility
Authentication	Login & Security
Learner	Profile Management
Brain Gym	Logical Practice
Aptitude Arena	Aptitude Assessment
Coding Arena	Coding Assessment
SQL Arena	SQL Assessment
Debug Hero	Debug Challenges
AI Coach	AI Learning Assistant
Resume Builder	Resume Management
Placement	Job & Placement
Recruiter	Company & Hiring
Gamification	XP & Rewards
Notification	Email/SMS/Push
Reports	Analytics & Audit
Shared	Common Utilities

3.7 Request Processing Flow

Client

↓

Controller

↓

Validation

↓

Service

↓

Repository

↓

Database

↓

Response DTO

↓

Client

Every request follows the same standardized lifecycle.

3.8 Cross-Cutting Concerns

These services are shared across all modules:

- Authentication
- Authorization
- Logging
- Exception Handling
- Validation

- Audit Trail
 - Notification
 - File Storage
 - AI Integration
 - Caching
-

3.9 Module Communication

Modules communicate through service interfaces.

Placement Module

↓

Resume Module

↓

AI Coach

↓

Notification Module

Rules:

- No direct database access between modules.
 - Use service interfaces for inter-module communication.
 - Shared utilities belong in the Shared module.
-

3.10 Shared Components

Common reusable components include:

- Base Entity
- Base DTO
- API Response Wrapper
- Global Exception Handler
- Validation Utilities
- Security Utilities
- File Upload Utility
- Pagination Support

- Audit Utility
 - Common Constants
-

3.11 Dependency Injection

Spring Boot manages dependencies using:

- Constructor Injection
- Service Injection
- Repository Injection
- Configuration Beans

Benefits:

- Loose Coupling
 - Easy Testing
 - Better Maintainability
-

3.12 Security Architecture

Security is applied at every layer.

Client

↓

JWT Authentication

↓

Spring Security

↓

Role Validation

↓

Permission Validation

↓

Business Logic

↓

Database

3.13 Caching Architecture

Redis is used for:

- Login Sessions
- OTP
- Dashboard Data
- Leaderboards
- AI Responses
- Frequently Used Master Data

Benefits:

- Faster Response Time
 - Reduced Database Load
 - Better Scalability
-

3.14 Asynchronous Processing

Background tasks include:

- Email Sending
 - SMS Delivery
 - Push Notifications
 - PDF Generation
 - AI Processing
 - Leaderboard Updates
 - Report Generation
 - Audit Processing
-

3.15 Exception Handling

A centralized exception handling framework manages:

- Validation Errors
- Business Exceptions
- Authentication Errors
- Authorization Errors
- Database Errors
- External Service Errors
- Unexpected System Errors

All API responses follow a consistent error format.

3.16 Logging & Audit

Every critical operation generates:

- Application Logs
- Audit Logs
- API Logs
- Security Logs
- Error Logs

Logs support troubleshooting, monitoring, and compliance.

3.17 Scalability Strategy

The architecture supports:

- Horizontal Scaling
 - Stateless APIs
 - Redis Caching
 - Database Read Replicas
 - Background Job Processing
 - Object Storage
 - Containerized Deployment
-

3.18 High Availability

High availability is achieved through:

- Load Balancers
 - Multiple Application Instances
 - PostgreSQL Replication
 - Redis Persistence
 - Automated Backups
 - Health Checks
 - Disaster Recovery
-

3.19 Design Principles

The platform follows:

- SOLID Principles
 - DRY (Don't Repeat Yourself)
 - KISS (Keep It Simple)
 - Separation of Concerns
 - Single Responsibility
 - Open/Closed Principle
 - Dependency Inversion
 - Interface Segregation
-

3.20 Enterprise Best Practices

- Keep business logic inside the service layer.
 - Controllers should remain lightweight.
 - Validate all incoming requests.
 - Use DTOs instead of exposing entities.
 - Apply transactions only where required.
 - Implement centralized logging.
 - Cache frequently accessed data.
 - Follow secure coding guidelines.
 - Write unit and integration tests.
 - Maintain comprehensive API documentation.
-

3.21 Chapter Summary

This chapter defines the **Enterprise Architecture** of the Smart Education Platform. It establishes a modular, layered, and clean architecture that separates presentation, business, persistence, and infrastructure concerns. By combining **Clean Architecture**, **Domain-Driven Design**, **SOLID principles**, centralized security, caching, logging, and scalable infrastructure, the platform is prepared for enterprise deployment while remaining flexible for future migration to microservices.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 4 – Clean Architecture, Layered Design, Module Organization & Dependency Management

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

4.1 Introduction

The **Smart Education Platform** follows **Clean Architecture** to create a highly maintainable, scalable, testable, and loosely coupled enterprise application.

Each layer has a well-defined responsibility, ensuring that business logic remains independent of frameworks, databases, and user interfaces. This architecture enables easier maintenance, faster development, improved testing, and a seamless future migration to microservices.

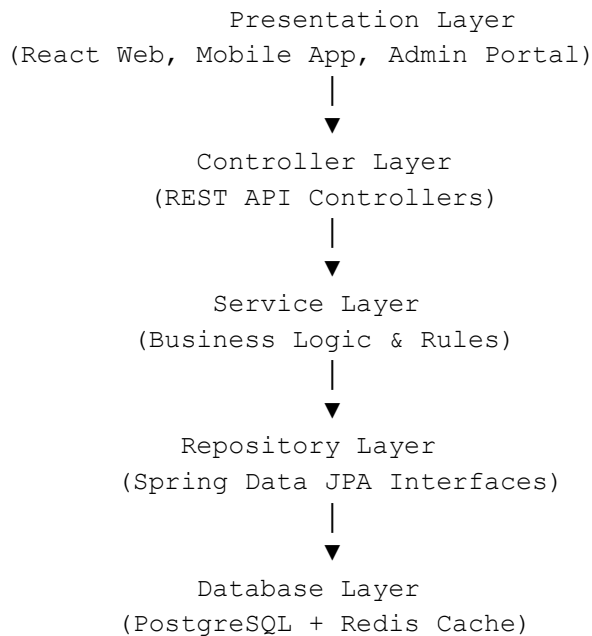
4.2 Objectives

The architecture is designed to achieve:

- Separation of Concerns
- High Cohesion
- Loose Coupling
- Independent Business Logic

- Framework Independence
 - Easy Testing
 - Code Reusability
 - Modular Development
 - Enterprise Scalability
-

4.3 Clean Architecture Overview



Dependency Rule: Dependencies always point inward. Business logic must never depend on external frameworks.

4.4 Layered Architecture

Layer 1 – Presentation Layer

Responsibilities:

- User Interface
- API Requests
- Authentication Token Management
- Form Validation
- Response Rendering

Technologies:

- React.js
 - React Router
 - Redux Toolkit
 - Axios
 - Tailwind CSS
-

Layer 2 – Controller Layer

Responsibilities:

- Receive HTTP Requests
- Validate Request DTOs
- Call Service Layer
- Return Standard API Responses
- Handle HTTP Status Codes

Controllers must **not** contain business logic.

Layer 3 – Service Layer

Responsibilities:

- Business Rules
- Transaction Management
- Workflow Coordination
- Validation of Business Policies
- Integration with Other Modules

Examples:

- Register Learner
 - Apply for Job
 - Generate Resume
 - Calculate XP
 - Evaluate Coding Submission
-

Layer 4 – Repository Layer

Responsibilities:

- CRUD Operations

- Database Queries
- Pagination
- Custom Queries
- Entity Persistence

Technologies:

- Spring Data JPA
 - Hibernate
-

Layer 5 – Database Layer

Responsibilities:

- Data Storage
- Constraints
- Indexes
- Stored Procedures
- Views
- Materialized Views

Technology:

- PostgreSQL 16+
-

4.5 Module Organization

The platform is organized into independent business modules.

`com.smarteducation`

```
|— authentication
|— learner
|— assessment
|   |— braingym
|   |— aptitude
|   |— coding
|   |— sql
|   |— debughero
|— gamification
|— ai
|— resume
|— placement
```

```
|— recruiter
|— notification
|— reports
|— shared
|— configuration
```

Each module owns its business logic and database access.

4.6 Internal Module Structure

Each module follows the same structure.

module

```
|— controller
|— service
|   |— impl
|— repository
|— entity
|— dto
|   |— request
|   |— response
|— mapper
|— validator
|— exception
|— specification
|— event
|— util
```

This consistent structure improves readability and maintainability.

4.7 Dependency Management

Dependencies follow strict rules:

Presentation

|

▼

Controller

|

▼

Service

|

▼

Repository



Forbidden dependencies:

- Controller → Repository
 - UI → Database
 - Repository → Controller
 - Module → Another Module's Database
-

4.8 Shared Components

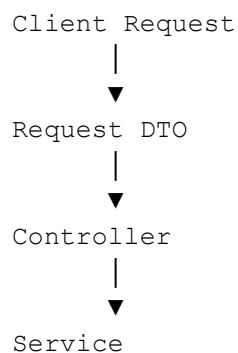
The `shared` module contains reusable components.

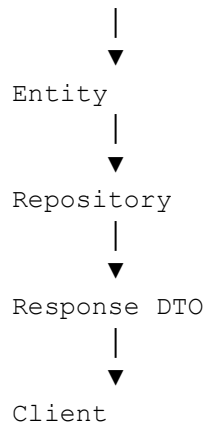
Examples:

- BaseEntity
 - BaseDTO
 - ApiResponse
 - Constants
 - Utility Classes
 - Exception Classes
 - Validation Utilities
 - Security Utilities
 - Pagination Models
 - Audit Helpers
-

4.9 DTO Architecture

Use DTOs for all API communication.





Benefits:

- Hide internal entities
 - Improve security
 - API versioning
 - Flexible response models
-

4.10 Entity Management

Entity classes should:

- Represent database tables
- Contain only persistence-related fields
- Avoid business logic
- Extend a common `BaseEntity` where appropriate

Common fields:

- `id`
 - `createdAt`
 - `updatedAt`
 - `createdBy`
 - `updatedBy`
 - `version`
-

4.11 Business Logic Guidelines

Business logic belongs only in the Service Layer.

Examples:

- Placement eligibility
- XP calculation
- Resume validation
- AI recommendation generation
- Interview scheduling

Controllers and repositories should remain lightweight.

4.12 Transaction Management

Transactions are managed at the Service Layer.

Examples:

- User Registration
- Job Application
- Resume Generation
- Offer Acceptance
- Reward Redemption

Guidelines:

- Keep transactions short.
 - Roll back on business failures.
 - Avoid long-running database transactions.
-

4.13 Exception Handling

A centralized exception handling framework manages:

- ValidationException
- BusinessException
- ResourceNotFoundException
- UnauthorizedException
- ForbiddenException
- DatabaseException
- ExternalServiceException

All exceptions return a standard API response format.

4.14 Logging Strategy

Every module generates:

- Application Logs
- Business Logs
- Audit Logs
- Security Logs
- Error Logs
- Performance Logs

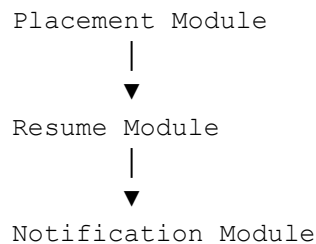
Logging should include:

- Request ID
 - User ID
 - Module
 - Execution Time
 - Error Details (if applicable)
-

4.15 Module Communication

Modules communicate through service interfaces.

Example:



Rules:

- No direct table access between modules.
 - Shared functionality belongs in the Shared module.
 - Module APIs should be stable and well documented.
-

4.16 Configuration Management

Application configuration includes:

- Database
- Redis
- JWT
- Email
- SMS
- AI Providers
- File Storage
- Logging
- Monitoring

Configurations should be externalized using profiles and environment variables.

4.17 Architectural Constraints

- No business logic inside controllers.
 - No SQL in controllers.
 - No entity exposure through REST APIs.
 - No circular module dependencies.
 - Validate all external inputs.
 - Use constructor injection.
 - Prefer interfaces over implementations.
-

4.18 Development Best Practices

- Follow SOLID principles.
 - Keep methods small and focused.
 - Write unit tests for services.
 - Document public APIs.
 - Reuse shared components.
 - Avoid duplicate business logic.
 - Apply consistent naming conventions.
 - Use code reviews before merging.
-

4.19 Benefits of This Architecture

- Easy to maintain
 - Highly testable
 - Modular and reusable
 - Secure by design
 - Scalable for future growth
 - Simplified onboarding for developers
 - Supports gradual migration to microservices
-

4.20 Chapter Summary

This chapter defines the **Clean Architecture, Layered Design, Module Organization, and Dependency Management** for the Smart Education Platform. It establishes clear responsibilities for each architectural layer, standardized module structures, strict dependency rules, centralized exception handling, reusable shared components, and enterprise development practices. This approach ensures that the platform remains maintainable, scalable, secure, and ready for future expansion.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 5 – Request Lifecycle, API Flow, Inter-Module Communication & Enterprise Integration Patterns

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

5.1 Introduction

Every request in the Smart Education Platform follows a standardized processing lifecycle to ensure **security, consistency, reliability, performance, and maintainability**.

This chapter defines how a request travels from the client application to the database and back, including authentication, validation, business processing, database transactions, caching, event publishing, and response generation.

5.2 Objectives

The request lifecycle is designed to provide:

- Secure API Processing
 - Standardized Request Flow
 - Consistent Responses
 - Transaction Safety
 - High Performance
 - Module Independence
 - Easy Debugging
 - Scalable Integration
-

5.3 End-to-End Request Lifecycle

Client (Web / Mobile)



Load Balancer / Nginx



Spring Security Filter



JWT Authentication



Authorization (RBAC)



REST Controller

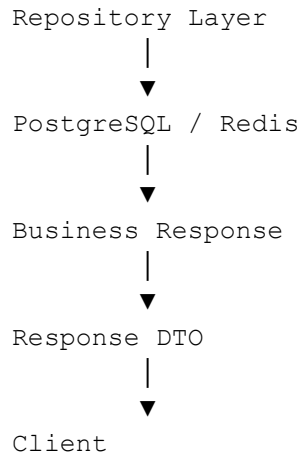


Request Validation



Service Layer



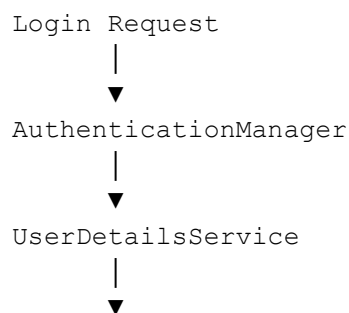


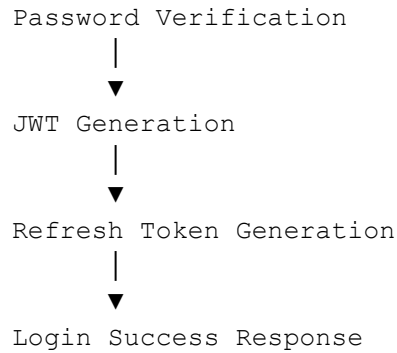
5.4 API Request Flow

Every API request follows these steps:

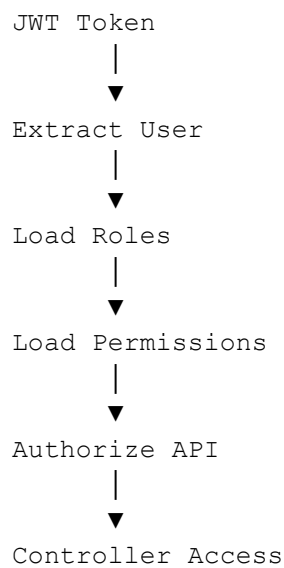
1. Client sends HTTP request.
 2. Nginx routes the request.
 3. Spring Security validates JWT.
 4. User permissions are verified.
 5. Request DTO validation is performed.
 6. Controller invokes the Service Layer.
 7. Business logic executes.
 8. Repository accesses the database.
 9. Transaction is committed or rolled back.
 10. Response DTO is returned.
 11. Audit log is created.
 12. Notifications/events are triggered if required.
-

5.5 Authentication Flow



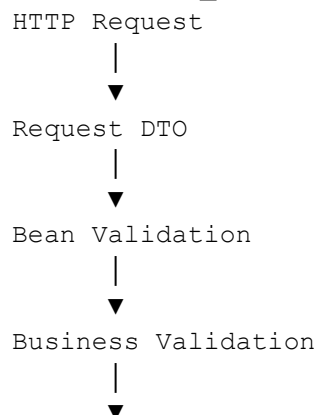


5.6 Authorization Flow



Only authorized users can access protected resources.

5.7 Request Validation Flow



Validation includes:

- Required fields
 - Email format
 - Mobile format
 - Password policy
 - Business constraints
-

5.8 Controller Processing

Controller responsibilities:

- Receive HTTP requests
- Validate DTOs
- Invoke services
- Return standardized responses
- Handle HTTP status codes

Controllers should never contain business logic.

5.9 Service Processing

The Service Layer performs:

- Business validation
- Workflow execution
- Transaction management
- Module coordination
- Event generation

Example services:

- Register Learner
 - Apply Job
 - Generate Resume
 - Submit Coding Solution
 - AI Resume Review
-

5.10 Repository Processing

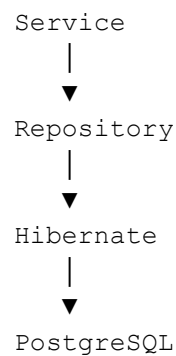
Repository responsibilities:

- CRUD operations
- Custom queries
- Pagination
- Database transactions
- Entity persistence

Technology:

- Spring Data JPA
 - Hibernate
-

5.11 Database Flow

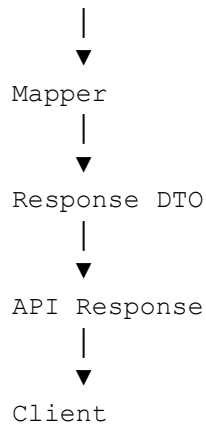


The database enforces:

- Constraints
 - Foreign Keys
 - Indexes
 - Transactions
 - Views
-

5.12 Response Flow





Responses never expose entity objects directly.

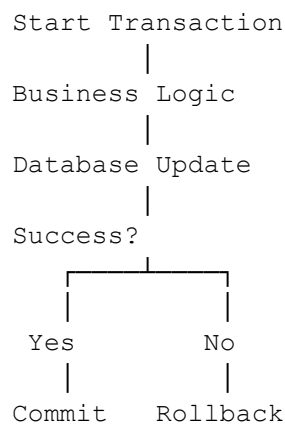
5.13 Standard API Response

Every API follows a common response structure.

```
{
  "success": true,
  "message": "Request processed successfully",
  "data": {},
  "timestamp": "2026-07-10T10:30:00Z"
}
```

Error responses follow the same structure with appropriate HTTP status codes.

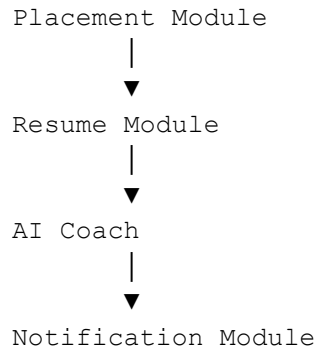
5.14 Transaction Lifecycle



Transactions are managed only at the Service Layer.

5.15 Inter-Module Communication

Modules communicate through service interfaces.



Rules:

- No direct database access across modules.
 - Communication through interfaces.
 - Shared functionality belongs to the Shared module.
-

5.16 Event-Driven Processing

Certain operations publish events.

Examples:

- User Registered
- Job Applied
- Coding Submission Evaluated
- Resume Generated
- Badge Awarded
- Offer Released

These events trigger:

- Notifications
 - Audit Logs
 - Analytics Updates
 - Gamification Updates
-

5.17 Asynchronous Processing

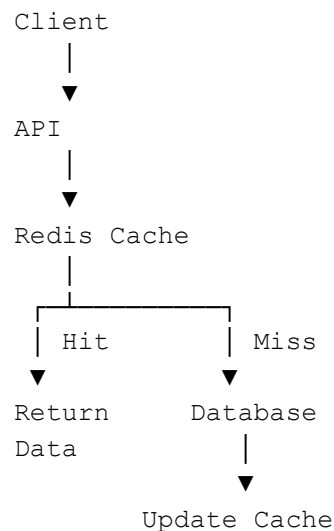
Background jobs include:

- Email sending
- SMS delivery
- Push notifications
- AI processing
- PDF generation
- Leaderboard calculation
- Report generation
- Data synchronization

Benefits:

- Faster API responses
 - Better scalability
 - Reduced user wait time
-

5.18 Caching Flow

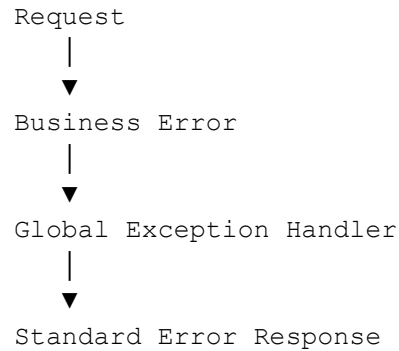


Frequently cached data:

- Master data
- Dashboard summaries
- Leaderboards
- User sessions

- OTPs
-

5.19 Exception Flow

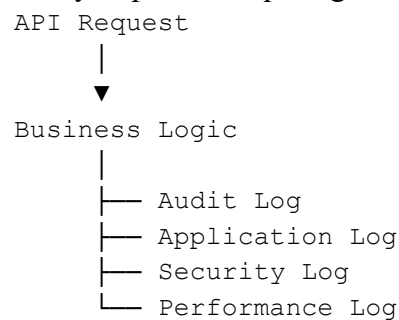


Exception types:

- ValidationException
 - BusinessException
 - AuthenticationException
 - AuthorizationException
 - ResourceNotFoundException
 - DatabaseException
-

5.20 Logging & Audit Flow

Every important request generates:



5.21 Performance Optimization

To improve request performance:

- Redis caching
 - Connection pooling (HikariCP)
 - Pagination
 - Database indexing
 - Batch processing
 - Asynchronous execution
 - Optimized SQL queries
 - Response compression
-

5.22 Security Considerations

- JWT authentication for protected APIs.
 - RBAC for authorization.
 - Input validation for all requests.
 - HTTPS for secure communication.
 - Rate limiting for public APIs.
 - Audit logging for sensitive operations.
 - Secure error handling without exposing internal details.
-

5.23 Enterprise Best Practices

- Keep controllers lightweight.
 - Centralize business logic in services.
 - Use DTOs for request and response objects.
 - Apply transactions only where necessary.
 - Cache frequently accessed data.
 - Log every critical operation.
 - Use asynchronous processing for long-running tasks.
 - Maintain API versioning for backward compatibility.
-

5.24 Chapter Summary

This chapter defines the **Request Lifecycle, API Flow, Inter-Module Communication, and Enterprise Integration Patterns** for the Smart Education Platform. It describes the complete journey of a request—from client to database and back—covering authentication, validation, business processing, transaction management, caching, event handling, logging, and standardized

responses. These patterns ensure a secure, scalable, high-performance, and maintainable enterprise application.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 6 – Security Architecture, Authentication, Authorization & Enterprise Security Model

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

6.1 Introduction

Security is one of the most critical aspects of the Smart Education Platform. The platform is designed using a **Security-by-Design** approach, where security is embedded into every architectural layer rather than added later.

The security model protects learners, recruiters, administrators, company data, AI services, assessments, resumes, and placement information from unauthorized access and cyber threats.

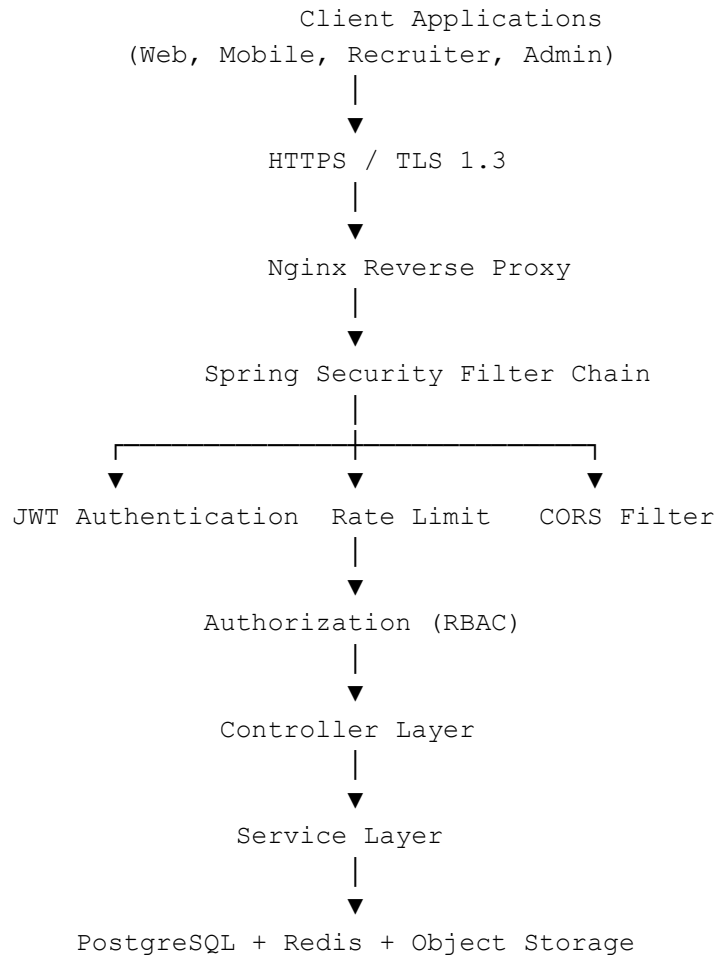
6.2 Security Objectives

The enterprise security architecture provides:

- Secure Authentication
- Role-Based Authorization
- Permission-Based Access Control
- API Security
- Data Encryption
- Session Security
- Secure File Access
- Audit Logging

- Threat Protection
 - Compliance Readiness
-

6.3 Enterprise Security Architecture



6.4 Security Layers

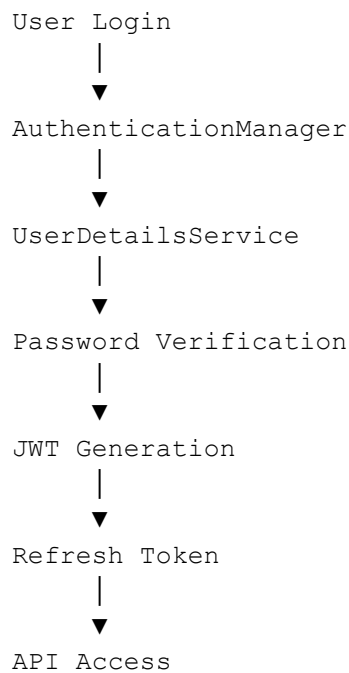
Layer	Responsibility
Network Security	HTTPS, Firewall, Reverse Proxy
Authentication	JWT Login
Authorization	RBAC & Permissions
API Security	Validation & Rate Limiting
Data Security	Encryption
Database Security	Roles & Access Control

Layer	Responsibility
Infrastructure Security	Docker & Server Hardening
Monitoring	Audit & Security Logs

6.5 Authentication Architecture

The platform uses **JWT-based Authentication**.

Authentication Process:



6.6 Login Flow

1. User enters email and password.
 2. Spring Security validates credentials.
 3. Password is verified using BCrypt.
 4. JWT Access Token is generated.
 5. Refresh Token is generated.
 6. User information is returned.
 7. All future requests use the JWT token.
-

6.7 JWT Authentication

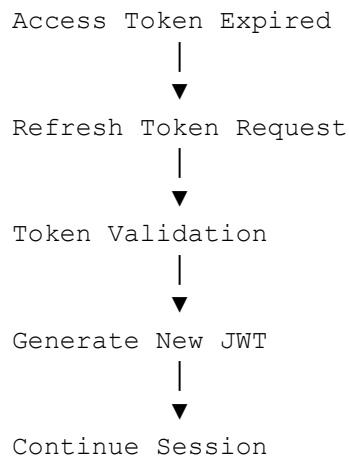
JWT contains:

- User ID
- Username
- Roles
- Permissions
- Token Issue Time
- Expiration Time

Recommended Configuration

- Access Token: **15–30 minutes**
 - Refresh Token: **7–30 days**
-

6.8 Refresh Token Strategy



Benefits:

- Improved security
 - Better user experience
 - Reduced login frequency
-

6.9 Authorization Model

The platform implements **Role-Based Access Control (RBAC)**.

Roles

- Super Admin
- Admin
- Trainer
- Learner
- Recruiter
- Company Admin
- Interviewer
- Support Executive

Each role has predefined permissions.

6.10 Permission-Based Access

Examples of permissions:

- USER_CREATE
 - USER_UPDATE
 - USER_DELETE
 - JOB_CREATE
 - JOB_UPDATE
 - JOB_APPROVE
 - RESUME_VIEW
 - RESUME_DOWNLOAD
 - REPORT_VIEW
 - REPORT_EXPORT
 - AI_ACCESS
 - SYSTEM_CONFIGURATION
-

6.11 Authorization Flow

Incoming Request

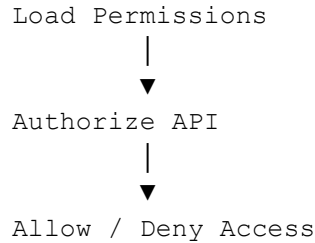


JWT Validation



Load User Roles





6.12 Spring Security Filter Chain

Request passes through:

- SecurityContext Filter
- JWT Authentication Filter
- UsernamePasswordAuthenticationFilter
- Authorization Filter
- Exception Translation Filter
- Controller

This ensures every protected request is authenticated before reaching business logic.

6.13 Password Security

Passwords must:

- Never be stored in plain text
- Be hashed using BCrypt
- Meet minimum complexity requirements
- Support password reset
- Support password expiry policies (optional)

Recommended Policy

- Minimum 8–12 characters
 - Uppercase letter
 - Lowercase letter
 - Number
 - Special character
-

6.14 API Security

Security measures include:

- HTTPS only
 - JWT authentication
 - Input validation
 - Output sanitization
 - Rate limiting
 - Request size limits
 - API versioning
 - Secure HTTP headers
-

6.15 Data Encryption

Encryption in Transit

- HTTPS
- TLS 1.3

Encryption at Rest

Encrypt:

- Refresh Tokens
 - Personal Information
 - Resume Files
 - Company Documents
 - Offer Letters
 - AI Configuration Secrets
-

6.16 File Security

Protected files include:

- Resume PDFs
- Certificates
- Company Documents

- Offer Letters
- Profile Photos

Security Features

- Private Object Storage
 - Signed URLs
 - Role-Based Downloads
 - Virus Scanning
 - File Size Validation
-

6.17 Session Security

Redis stores:

- Refresh Tokens
- OTPs
- Temporary Sessions
- Blacklisted JWT Tokens

Benefits

- Fast validation
 - Immediate logout support
 - Centralized session management
-

6.18 Security Logging

Log all security-related events:

- Login
- Logout
- Password Change
- Failed Login
- Permission Denied
- Token Refresh
- User Lock
- Role Changes
- Sensitive Data Access

6.19 Audit Trail

Audit every critical business action:

- User Creation
- Profile Update
- Job Posting
- Resume Download
- Interview Scheduling
- Offer Release
- AI Configuration Changes
- System Configuration Updates

Each audit record should capture:

- User ID
 - Action
 - Module
 - Timestamp
 - IP Address
 - Previous Value
 - New Value
-

6.20 Threat Protection

Protection mechanisms:

- SQL Injection Prevention
 - XSS Protection
 - CSRF Protection (where applicable)
 - Clickjacking Protection
 - Brute Force Protection
 - Rate Limiting
 - Input Sanitization
 - Secure HTTP Headers
-

6.21 Security Monitoring

Monitor:

- Failed Login Attempts
- Suspicious API Requests
- Unauthorized Access
- High Error Rates
- Token Abuse
- Excessive Downloads
- Privilege Escalation Attempts

Alerts should be generated for critical events.

6.22 Secure Coding Standards

Developers should:

- Validate all inputs.
 - Use parameterized queries.
 - Avoid exposing stack traces.
 - Never hardcode secrets.
 - Store credentials securely.
 - Apply least privilege.
 - Sanitize user inputs.
 - Use secure random number generators.
-

6.23 Compliance & Best Practices

The platform should align with recognized security practices such as:

- OWASP Top 10
- Secure Password Storage
- Principle of Least Privilege
- Defense in Depth
- Security by Design
- Regular Vulnerability Assessments
- Periodic Security Audits

6.24 Enterprise Security Checklist

- HTTPS enabled
 - JWT authentication configured
 - Refresh token rotation implemented
 - RBAC enforced
 - API rate limiting enabled
 - Audit logging enabled
 - Database encryption configured
 - File access secured
 - Secrets managed outside source code
 - Regular backups verified
-

6.25 Chapter Summary

This chapter defines the **Security Architecture, Authentication, Authorization, and Enterprise Security Model** for the Smart Education Platform. It establishes a comprehensive security framework using **Spring Security, JWT authentication, Role-Based Access Control (RBAC)**, secure session management, encryption, audit logging, and enterprise security best practices. These measures provide a strong foundation for protecting users, business data, APIs, and infrastructure while supporting secure, scalable, and compliant enterprise operations.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 7 – Database Architecture, Persistence Layer, ORM Strategy & Data Access Design

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Database: PostgreSQL 16+

7.1 Introduction

The **Database Architecture** provides the foundation for storing, managing, and retrieving application data securely and efficiently. The persistence layer is designed using **Spring Data JPA** and **Hibernate ORM**, ensuring clean separation between business logic and database operations.

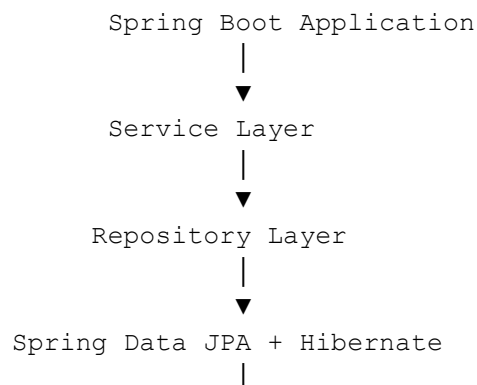
The architecture supports high-performance transactions, scalability, caching, auditing, and future migration to distributed systems.

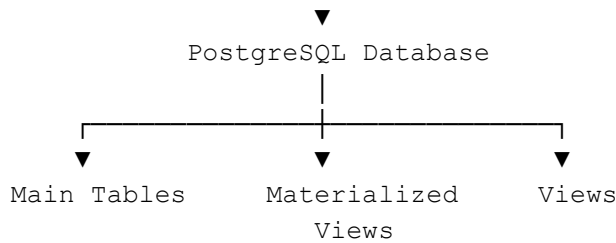
7.2 Objectives

The database architecture is designed to provide:

- High Performance
 - ACID Compliance
 - Data Integrity
 - Scalability
 - Maintainability
 - Secure Data Access
 - Efficient ORM Mapping
 - Transaction Management
 - Query Optimization
 - Enterprise Reporting
-

7.3 Database Architecture Overview



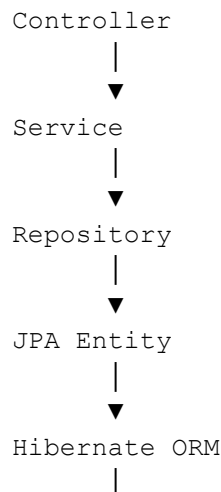


7.4 Database Schema Organization

The database is divided into logical schemas.

Schema	Purpose
auth	Authentication & Security
learner	Learner Management
assessment	Brain Gym, Aptitude, Coding, SQL, Debug Hero
ai	AI Coach
resume	Resume Builder
placement	Placement Portal
recruiter	Recruiter Portal
gamification	XP, Coins & Badges
notification	Email, SMS & Push
report	Reports & Analytics
audit	Audit Logs

7.5 Persistence Layer Architecture



▼
PostgreSQL

Responsibilities:

- Entity persistence
 - CRUD operations
 - Custom queries
 - Transaction support
 - Pagination
 - Sorting
-

7.6 ORM Strategy

The platform uses **Hibernate ORM** to map Java objects to relational tables.

Advantages:

- Reduced boilerplate code
 - Automatic SQL generation
 - Database independence
 - Lazy/Eager loading
 - Built-in caching
 - Transaction management
-

7.7 Entity Design Standards

Each entity:

- Represents one database table.
- Uses `@Entity` and `@Table`.
- Uses `BIGSERIAL/BIGINT` primary keys.
- Extends `BaseEntity` where applicable.
- Includes audit fields.

Example common fields:

- `id`
- `createdAt`
- `updatedAt`

- createdBy
 - updatedBy
 - version
-

7.8 Repository Layer Design

Repositories extend Spring Data JPA interfaces.

Examples:

- JpaRepository
- PagingAndSortingRepository
- JpaSpecificationExecutor

Repository responsibilities:

- CRUD operations
 - Search
 - Pagination
 - Dynamic filtering
 - Custom JPQL/SQL queries
-

7.9 Entity Relationships

Supported mappings:

- One-to-One
- One-to-Many
- Many-to-One
- Many-to-Many

Best Practices:

- Prefer `LAZY` loading by default.
 - Use `EAGER` only when necessary.
 - Avoid unnecessary bidirectional relationships.
-

7.10 Transaction Management

Transactions are managed in the Service Layer using Spring's `@Transactional`.

Examples:

- User Registration
- Resume Creation
- Job Application
- Interview Scheduling
- Reward Redemption

Guidelines:

- Keep transactions short.
 - Roll back on business exceptions.
 - Avoid nested transactions where possible.
-

7.11 Query Strategy

Use the following query types:

- Derived Query Methods
- JPQL
- Native SQL
- Specifications API
- Criteria API (for complex queries)

Optimization techniques:

- Pagination
 - Indexed searches
 - Batch updates
 - Fetch joins where appropriate
-

7.12 Pagination & Sorting

Large datasets must support:

- Page Number
- Page Size
- Sorting

- Dynamic Filtering

Benefits:

- Reduced memory usage
 - Faster API responses
 - Improved user experience
-

7.13 Auditing

Enable automatic auditing using Spring Data JPA.

Track:

- Created By
- Created At
- Updated By
- Updated At

Benefits:

- Data traceability
 - Compliance
 - Operational visibility
-

7.14 Caching Integration

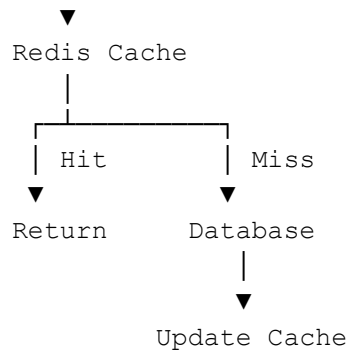
Redis is integrated with the persistence layer.

Cached data includes:

- Master Data
- User Profiles
- Dashboard Data
- Leaderboards
- Frequently Used Settings

Cache Strategy:

Request
|



7.15 Database Performance Optimization

Optimization techniques:

- Proper indexing
 - Query tuning
 - Connection pooling (HikariCP)
 - Batch inserts and updates
 - Materialized views
 - Partitioning for large tables
 - Avoid unnecessary joins
-

7.16 Connection Pooling

The application uses **HikariCP**.

Benefits:

- Faster database access
 - Reduced connection overhead
 - Efficient resource utilization
 - Improved scalability
-

7.17 Exception Handling

Database-related exceptions include:

- `DuplicateKeyException`

- `DataIntegrityViolationException`
- `OptimisticLockException`
- `ConstraintViolationException`
- `TransactionException`

All exceptions are handled by a centralized Global Exception Handler.

7.18 Database Security

Security measures include:

- Parameterized queries
 - Prepared statements
 - Role-based database access
 - Encrypted connections (TLS)
 - Audit logging
 - Sensitive data encryption
-

7.19 Best Practices

- Use DTOs instead of exposing entities.
 - Prefer constructor injection.
 - Avoid business logic in repositories.
 - Keep entities lightweight.
 - Use optimistic locking for concurrent updates.
 - Minimize N+1 query issues.
 - Monitor slow queries regularly.
-

7.20 Future Enhancements

Future improvements may include:

- Read replicas for reporting
- Multi-tenant database architecture
- Database sharding
- Event sourcing
- CQRS (Command Query Responsibility Segregation)

- Reactive database support (R2DBC)
 - AI-assisted query optimization
-

7.21 Chapter Summary

This chapter defines the **Database Architecture, Persistence Layer, ORM Strategy, and Data Access Design** for the Smart Education Platform. It establishes a robust persistence model using **PostgreSQL, Spring Data JPA, and Hibernate**, along with transaction management, repository patterns, caching, auditing, and performance optimization. The architecture provides a secure, scalable, and maintainable foundation for enterprise-grade data management.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 8 – Caching Architecture, Redis, File Storage, Object Storage & Media Management

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Cache: Redis 7+

Object Storage: MinIO / AWS S3

8.1 Introduction

To deliver high performance and enterprise scalability, the Smart Education Platform employs a dedicated caching and storage architecture.

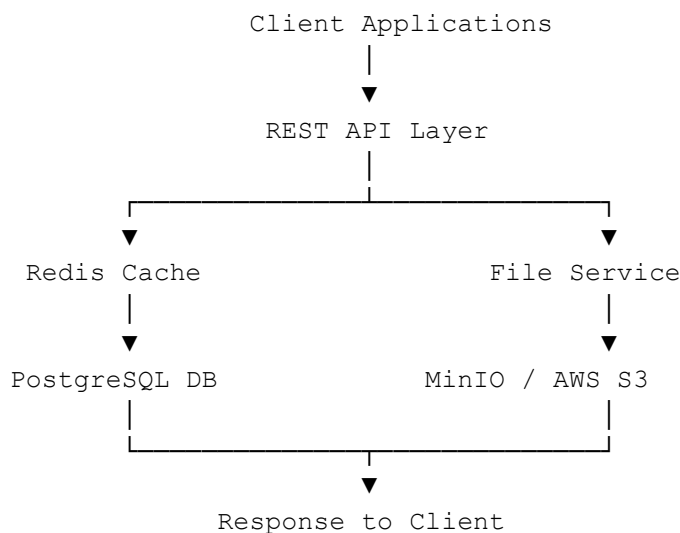
The architecture combines **Redis** for high-speed in-memory caching and **Object Storage (MinIO/AWS S3)** for scalable document and media management. This reduces database load, improves API response times, enables efficient file handling, and supports future cloud-native deployments.

8.2 Objectives

The architecture is designed to provide:

- High-Speed Data Access
 - Reduced Database Load
 - Secure File Storage
 - Scalable Object Storage
 - Media Management
 - Distributed Session Management
 - Backup & Recovery
 - CDN Readiness
 - Enterprise Performance
 - Cloud Compatibility
-

8.3 Architecture Overview



8.4 Redis Architecture

Redis acts as the primary in-memory cache.

Application

|

Redis

|

User Cache

OTP Cache

Session Cache

Dashboard Cache

Leaderboard Cache

AI Cache

Master Data Cache

8.5 Redis Use Cases

Redis stores:

- User Sessions
 - JWT Blacklist
 - Refresh Tokens
 - OTP Verification
 - Frequently Accessed Master Data
 - Dashboard Statistics
 - AI Responses
 - Leaderboards
 - Rate Limiting Counters
 - Temporary Processing Data
-

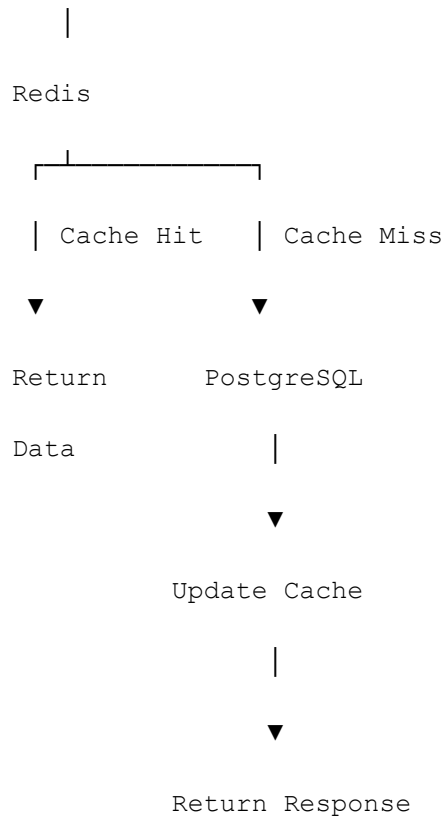
8.6 Cache Strategy

The platform primarily uses the **Cache-Aside Pattern**.

Client

|

API



Benefits:

- Faster Response
- Lower Database Load
- Better Scalability

8.7 Cache Categories

Cache Type	Purpose
Session Cache	User Sessions
Security Cache	JWT & Refresh Tokens
Dashboard Cache	Dashboard Widgets
Master Data Cache	Lookup Tables
Leaderboard Cache	Rankings
AI Cache	AI Responses
Configuration Cache	System Settings

8.8 Cache Expiration Strategy

Data	TTL
OTP	5 Minutes
JWT Blacklist	Token Expiry Time
Dashboard	5 Minutes
Master Data	12 Hours
AI Responses	30 Minutes
Leaderboards	10 Minutes
System Configuration	24 Hours

8.9 Cache Invalidation

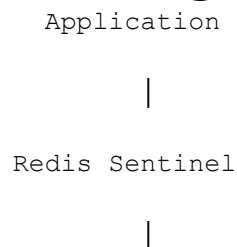
Cache is refreshed when:

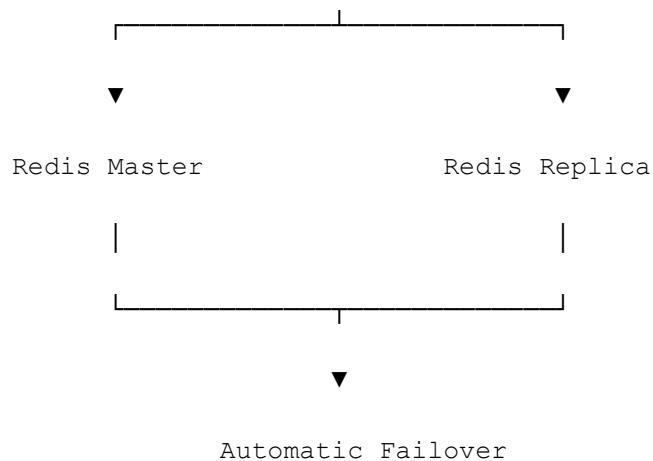
- User Profile Updated
- Resume Updated
- Job Posted
- Assessment Completed
- Leaderboard Updated
- AI Recommendation Generated
- Settings Modified

Invalidation Methods:

- Time-Based Expiration
 - Manual Eviction
 - Event-Based Refresh
-

8.10 Redis High Availability





8.11 Session Management

Redis manages:

- Login Sessions
- Refresh Tokens
- OTP Sessions
- Password Reset Tokens
- Email Verification Tokens

Benefits:

- Centralized Session Storage
- Horizontal Scaling
- Fast Session Lookup
- Immediate Logout Support

8.12 File Storage Architecture

Files are stored outside the database.

Application

Upload API

File Validation



Object Storage



Metadata Database



Download URL

Only metadata is stored in PostgreSQL.

8.13 Object Storage

Recommended Options:

- MinIO (Self-Hosted)
 - AWS S3
 - Azure Blob Storage (Future)
 - Google Cloud Storage (Future)
-

8.14 File Categories

Category	Examples
Profile Images	JPG, PNG
Resume Files	PDF
Certificates	PDF, JPG
Company Logos	PNG, SVG
Offer Letters	PDF
AI Generated Reports	PDF
Course Resources	PDF, ZIP
Videos	MP4
Documents	DOCX, XLSX, PPTX

8.15 File Upload Flow

Client

|

Upload Request

|

Validation

|

Virus Scan

|

Generate Unique File Name

|

Store in Object Storage

|

Save Metadata

|

Return File URL

8.16 File Metadata

Database stores:

- File ID
- File Name
- Original Name
- File Type
- MIME Type
- File Size
- Storage Path
- Bucket Name

- Uploaded By
 - Upload Time
 - Checksum
 - Status
-

8.17 Media Processing

Supported operations:

- Image Resize
 - Image Compression
 - Thumbnail Generation
 - PDF Preview Generation
 - Video Thumbnail Extraction
 - File Compression
 - Metadata Extraction
-

8.18 Secure File Access

Protected files include:

- Resume PDFs
- Company Documents
- Certificates
- Offer Letters
- AI Reports

Security Features:

- Signed URLs
 - Token-Based Downloads
 - Role-Based Authorization
 - Download Expiration
 - Download Audit Logs
-

8.19 CDN Integration

Future architecture:

Client

|

CDN

|

Object Storage

|

Application

Benefits:

- Faster Downloads
- Global Distribution
- Reduced Server Load
- Better Performance

8.20 Backup Strategy

Object Storage Backup:

- Daily Incremental Backup
- Weekly Full Backup
- Cross-Region Replication (Future)
- Encrypted Backup Storage

Redis Backup:

- RDB Snapshots
- Append Only File (AOF)
- Automatic Recovery

8.21 Storage Monitoring

Monitor:

- Storage Utilization
 - Upload Rate
 - Download Rate
 - Failed Uploads
 - Failed Downloads
 - Cache Hit Ratio
 - Cache Miss Ratio
 - Redis Memory Usage
 - Object Storage Capacity
-

8.22 Performance Optimization

Performance techniques:

- Redis Connection Pooling
 - Lazy Cache Loading
 - Cache Preloading
 - File Compression
 - CDN Caching
 - Asynchronous Upload Processing
 - Streaming Downloads
 - Large File Chunk Uploads
-

8.23 Security Best Practices

- Encrypt files at rest.
 - Encrypt file transfers using HTTPS.
 - Validate MIME types.
 - Restrict maximum upload size.
 - Scan uploaded files for malware.
 - Use signed URLs for downloads.
 - Apply bucket-level access control.
 - Maintain complete file audit logs.
-

8.24 Future Enhancements

Future improvements may include:

- Multi-Region Object Storage
 - AI-Based Image Optimization
 - Intelligent Cache Warming
 - Distributed Redis Cluster
 - Video Streaming Support
 - Media Transcoding
 - Automatic File Archival
 - Lifecycle Management Policies
 - Cloud CDN Integration
 - Edge Caching
-

8.25 Chapter Summary

This chapter defines the **Caching Architecture, Redis, File Storage, Object Storage, and Media Management** strategy for the Smart Education Platform. It outlines how Redis improves application performance through intelligent caching and session management, while MinIO/AWS S3 provides scalable, secure, and reliable object storage for documents and media. Together, these components reduce database load, improve response times, enhance scalability, and provide enterprise-grade file management.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 9 – AI Architecture, External Service Integration, Event-Driven Architecture & Enterprise Integration Patterns

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

AI Framework: Spring AI

9.1 Introduction

Artificial Intelligence is one of the core pillars of the Smart Education Platform. The platform uses AI to deliver personalized learning experiences, intelligent career guidance, resume analysis, interview preparation, coding assistance, SQL optimization, and learning recommendations.

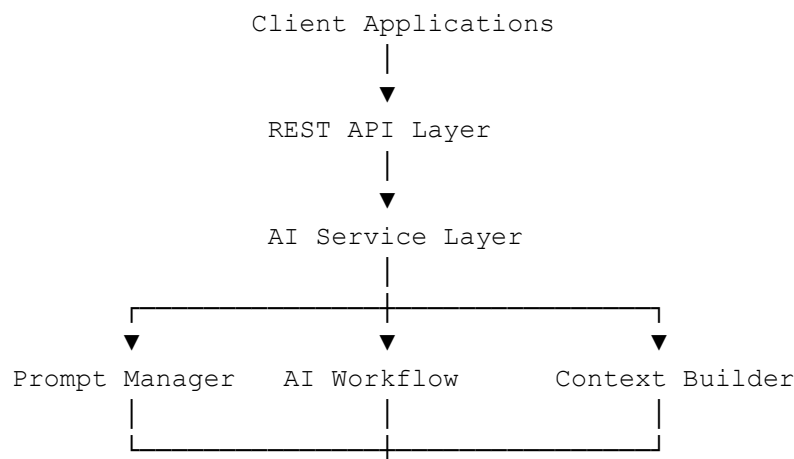
This chapter defines the complete AI architecture, external service integration strategy, event-driven communication model, and enterprise integration patterns that enable seamless interaction between internal modules and third-party services.

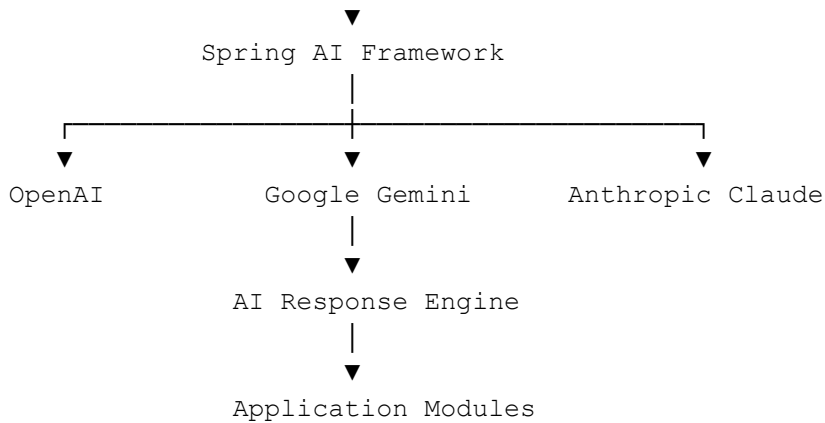
9.2 Objectives

The architecture is designed to provide:

- AI-Powered Learning
 - Multi-LLM Support
 - Secure External Integrations
 - Event-Driven Processing
 - Scalable AI Services
 - Fault-Tolerant Communication
 - Asynchronous Processing
 - Provider Independence
 - Future Microservice Compatibility
 - Enterprise Integration Standards
-

9.3 AI Architecture Overview





9.4 AI Service Architecture

The AI Service Layer is responsible for:

- Prompt Management
- Context Building
- Model Selection
- Response Processing
- Usage Tracking
- Token Management
- Error Handling
- AI Analytics

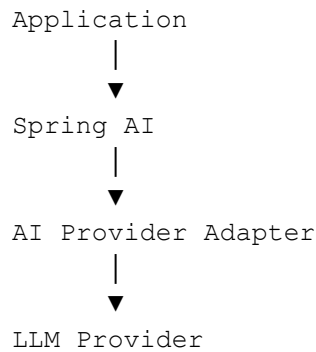
9.5 AI Modules

The platform provides the following AI capabilities:

AI Module	Purpose
AI Coach	General Assistance
Resume AI	Resume Review
Interview AI	Mock Interview
Coding AI	Coding Assistance
SQL AI	SQL Optimization
Career AI	Career Guidance
Learning AI	Personalized Learning Path
Analytics AI	Learning Insights

9.6 Spring AI Architecture

Spring AI acts as the abstraction layer between the application and AI providers.



Benefits

- Provider Independence
- Unified API
- Easy Provider Switching
- Future Extensibility

9.7 Multi-LLM Support

Supported providers include:

- OpenAI
- Google Gemini
- Anthropic Claude
- Azure OpenAI
- Local LLMs (Future)

The architecture allows dynamic selection of the provider based on:

- Feature
 - Cost
 - Performance
 - Availability
 - Organization Policy
-

9.8 Prompt Management

Prompt templates are stored separately from business logic.

Example Categories

- Resume Review
- Interview Evaluation
- Coding Hint
- SQL Query Optimization
- Career Advice
- Study Plan
- Job Recommendation

Prompt Features

- Versioning
 - Variable Substitution
 - Reusability
 - Localization (Future)
-

9.9 Context Builder

Before sending a request to the AI provider, the Context Builder collects relevant information.

Examples

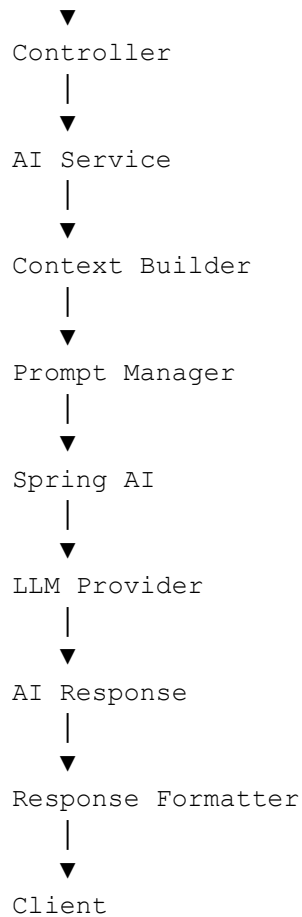
- Learner Profile
- Learning Progress
- Resume Data
- Assessment Scores
- Job Preferences
- Previous Conversations

Only required information is included to optimize token usage.

9.10 AI Request Flow

Client

|



9.11 AI Response Processing

Responses are processed through:

- Validation
- Content Filtering
- Formatting
- Token Tracking
- Audit Logging
- Analytics Collection

This ensures consistent and secure AI outputs.

9.12 AI Usage Tracking

Track the following metrics:

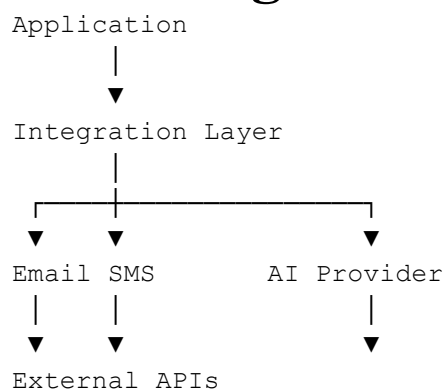
- Total Requests
- Total Responses
- Prompt Tokens
- Completion Tokens
- Total Tokens
- Response Time
- Provider Used
- Feature Used
- Estimated Cost

9.13 External Service Integration

The platform integrates with multiple external services.

Service	Purpose
AI Providers	AI Capabilities
SMTP Server	Email
SMS Gateway	SMS Delivery
Push Notification Provider	Mobile Notifications
Object Storage	File Storage
Payment Gateway (Future)	Payments
Video Meeting Platform	Interviews
Maps API (Future)	Location Services

9.14 Integration Architecture



The Integration Layer isolates business logic from third-party services.

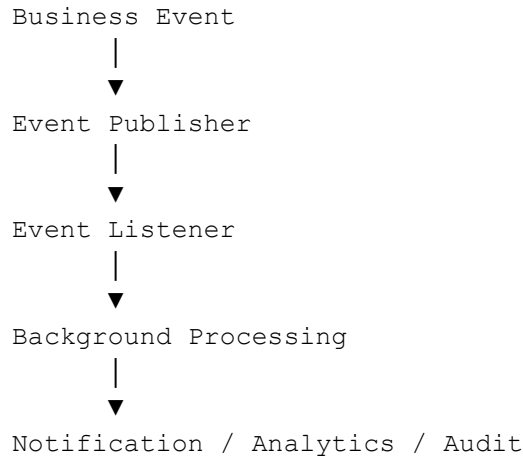
9.15 Event-Driven Architecture

Business events trigger asynchronous processing.

Examples

- User Registered
 - Job Applied
 - Resume Uploaded
 - Assessment Completed
 - AI Review Completed
 - Interview Scheduled
 - Offer Released
 - Badge Awarded
-

9.16 Event Processing Flow



9.17 Enterprise Integration Patterns

Supported patterns:

- Request-Response
- Publish-Subscribe
- Event Notification

- Retry Pattern
- Circuit Breaker
- Adapter Pattern
- Facade Pattern
- Strategy Pattern
- Factory Pattern

These patterns improve reliability and maintainability.

9.18 Retry Strategy

Transient failures are retried automatically.

Recommended Policy

- Maximum Retries: 3
- Exponential Backoff
- Configurable Timeout
- Dead Letter Queue (Future)

Applicable Services

- AI Provider
 - Email
 - SMS
 - File Upload
-

9.19 Fault Tolerance

The platform provides:

- Timeout Handling
 - Retry Mechanism
 - Fallback Responses
 - Provider Switching
 - Graceful Degradation
 - Error Logging
-

9.20 Message Queue Readiness

Current Architecture

- Spring Events
- Async Processing

Future Upgrade

- Apache Kafka
- RabbitMQ
- Amazon SQS

Possible Event Topics

- Notifications
 - AI Processing
 - Assessment Results
 - Placement Events
 - Analytics Updates
-

9.21 Webhook Architecture

External systems can receive notifications through secure webhooks.

Supported Events

- Job Application Submitted
- Interview Scheduled
- Offer Released
- Resume Updated
- Assessment Completed

Webhook Features

- HTTPS
 - Authentication
 - Retry
 - Signature Verification
 - Delivery Logs
-

9.22 Integration Security

Security measures include:

- HTTPS Communication
 - API Keys
 - OAuth 2.0 (where supported)
 - JWT Authentication
 - Rate Limiting
 - Request Validation
 - Secret Management
 - Audit Logging
-

9.23 Monitoring & Observability

Monitor:

- AI Response Time
- External API Latency
- Success Rate
- Failure Rate
- Retry Count
- Token Consumption
- Provider Availability
- Integration Errors

Alerts should be configured for repeated failures.

9.24 Future Enhancements

Future capabilities may include:

- AI Agent Framework
- Retrieval-Augmented Generation (RAG)
- Vector Database Integration
- AI Memory Management
- Multi-Agent Collaboration
- Real-Time Voice AI

- AI Workflow Orchestration
 - Intelligent Routing Across LLMs
 - Event Streaming with Kafka
 - Enterprise Service Bus (ESB) Integration
-

9.25 Chapter Summary

This chapter defines the **AI Architecture, External Service Integration, Event-Driven Architecture, and Enterprise Integration Patterns** for the Smart Education Platform. It establishes a flexible AI layer built on **Spring AI**, enabling support for multiple LLM providers while maintaining provider independence. The chapter also outlines secure integration with external services, asynchronous event processing, fault tolerance, monitoring, and enterprise integration patterns, ensuring a scalable, resilient, and AI-ready platform architecture.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 10 – Notification Architecture, Scheduler, Background Jobs, Automation & Workflow Engine

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Framework: Spring Boot 3.x

Scheduler: Spring Scheduler

Async Processing: Spring Async

10.1 Introduction

The **Notification, Scheduler, Background Job, Automation, and Workflow Engine** architecture enables the Smart Education Platform to execute long-running processes

asynchronously while providing real-time communication with learners, recruiters, administrators, and external systems.

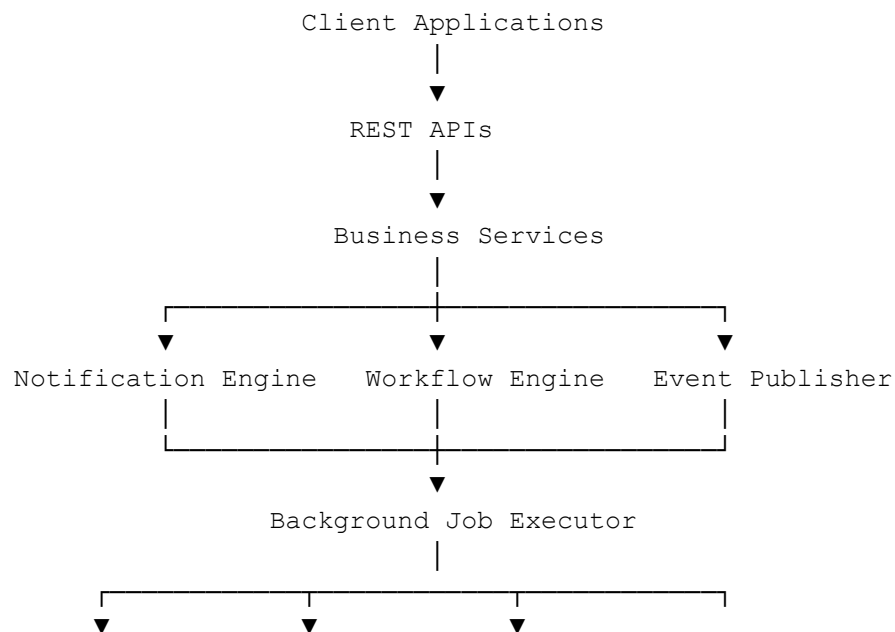
Instead of executing every operation during an API request, time-consuming tasks are delegated to background workers, improving system performance, scalability, and user experience.

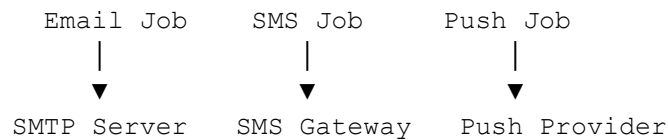
10.2 Objectives

The architecture is designed to provide:

- Email Notifications
 - SMS Notifications
 - Push Notifications
 - In-App Notifications
 - Background Processing
 - Scheduled Jobs
 - Workflow Automation
 - Event-Based Processing
 - Retry Mechanisms
 - Fault Tolerance
 - Enterprise Scalability
-

10.3 Overall Architecture





10.4 Notification Architecture

The Notification Engine is responsible for:

- Email Delivery
 - SMS Delivery
 - Push Notifications
 - In-App Notifications
 - Notification Preferences
 - Retry Handling
 - Delivery Tracking
 - Template Processing
-

10.5 Notification Types

Type	Description
Email	Transactional emails
SMS	OTP & alerts
Push Notification	Mobile notifications
In-App Notification	Dashboard notifications
System Notification	Administrative alerts

10.6 Notification Categories

Examples include:

Authentication

- Registration
- Login Alert
- Password Reset
- OTP Verification

Assessment

- Test Scheduled
- Test Reminder
- Result Published
- Certificate Available

Placement

- Job Posted
- Application Submitted
- Interview Scheduled
- Offer Released
- Joining Reminder

AI Coach

- Resume Review Completed
- AI Recommendation Ready
- Mock Interview Feedback

Gamification

- XP Earned
- Badge Unlocked
- Challenge Completed
- Leaderboard Updated

10.7 Notification Flow

Business Event

|

Event Publisher

|

Notification Engine

|

Template Engine

|
Delivery Service

|
User

10.8 Email Service

Email capabilities:

- Registration Email
- Password Reset
- Verification Email
- Placement Notifications
- Resume Feedback
- Interview Schedule
- Offer Letter Notification
- Weekly Reports

Technology

- Spring Mail
 - SMTP
 - HTML Templates
-

10.9 SMS Service

SMS is used for:

- OTP
 - Password Reset
 - Login Verification
 - Interview Reminder
 - Placement Alert
 - Emergency Notification
-

10.10 Push Notification Service

Push notifications support:

- Android
- iOS
- Web Browser (Future)

Examples

- Assignment Reminder
 - AI Feedback Ready
 - Challenge Available
 - Job Recommendation
 - Interview Reminder
-

10.11 In-App Notification Center

Users receive notifications inside the application.

Features

- Unread Count
 - Mark as Read
 - Notification History
 - Category Filter
 - Search
 - Real-Time Updates
-

10.12 Notification Template Engine

Templates are maintained separately.

Example Templates

- Welcome Email
- OTP
- Password Reset
- Interview Invitation

- Placement Offer
- Badge Earned
- AI Report Ready

Template Variables

{{name}}

{{company}}

{{job}}

{{otp}}

{{date}}

10.13 Background Job Architecture

API Request

|

Business Service

|

Background Queue

|

Async Worker

|

Task Execution

Background execution keeps API response times low.

10.14 Background Jobs

Examples

- Email Sending
- SMS Sending

- Push Notification
 - Resume PDF Generation
 - AI Processing
 - Report Generation
 - Leaderboard Calculation
 - ATS Score Calculation
 - Backup Processing
 - File Cleanup
-

10.15 Scheduler Architecture

Spring Scheduler executes recurring tasks.

Cron Scheduler

|

Scheduled Task

|

Business Service

|

Database

|

Notification

10.16 Scheduled Jobs

Daily Jobs

- Leaderboard Refresh
- Report Generation
- Cache Cleanup
- Session Cleanup

Weekly Jobs

- Database Maintenance
- Analytics Summary
- Backup Validation

Monthly Jobs

- Archive Old Logs
 - Generate Monthly Reports
 - Capacity Analysis
-

10.17 Workflow Engine

Workflow engine manages business processes.

Examples

Job Application

Apply

↓

Shortlist

↓

Interview

↓

Offer

↓

Joined

Resume Review

Upload Resume

↓

AI Analysis

↓

ATS Score

↓

Suggestions

↓

Completed

Coding Assessment

Submit Code

↓

Compile

↓

Execute

↓

Evaluate

↓

Leaderboard

10.18 Automation Rules

Examples

When:

User Registers

Then:

- Send Welcome Email
- Assign Default Role
- Create Dashboard
- Award Welcome XP

When:

Assessment Completed

Then:

- Calculate Score
 - Award XP
 - Update Leaderboard
 - Generate Certificate
 - Notify User
-

When:

Offer Released

Then:

- Notify Learner
 - Notify Recruiter
 - Update Placement Status
 - Generate Analytics
-

10.19 Retry Strategy

Failed jobs are retried automatically.

Configuration

- Maximum Retries: 3
- Exponential Backoff
- Retry Delay
- Failure Logging

Retry applies to:

- Email
- SMS
- Push

- AI Requests
 - External APIs
-

10.20 Dead Letter Processing

If retries fail:

Failed Job

↓

Retry

↓

Retry

↓

Retry

↓

Dead Letter Queue

↓

Administrator Review

This prevents endless retry loops.

10.21 Job Monitoring

Monitor:

- Running Jobs
 - Failed Jobs
 - Pending Jobs
 - Completed Jobs
 - Retry Count
 - Execution Time
 - Queue Length
-

10.22 Notification Preferences

Users can configure:

- Email Notifications
 - SMS Notifications
 - Push Notifications
 - Marketing Notifications
 - Placement Alerts
 - AI Notifications
 - Reminder Frequency
-

10.23 Performance Optimization

Techniques

- Async Processing
 - Batch Notifications
 - Template Caching
 - Redis Queue Support
 - Connection Pooling
 - Parallel Processing
 - Scheduled Cache Refresh
-

10.24 Security Considerations

- Validate notification recipients.
 - Encrypt sensitive notification data where required.
 - Never include passwords in notifications.
 - Protect API keys for SMS and email providers.
 - Log notification delivery status.
 - Apply rate limits to notification APIs.
 - Allow users to manage notification preferences.
-

10.25 Enterprise Best Practices

- Keep notifications asynchronous.
 - Use reusable templates.
 - Separate notification logic from business logic.
 - Implement centralized retry handling.
 - Monitor delivery success rates.
 - Archive old notification records.
 - Design workflows to be idempotent where possible.
 - Maintain audit logs for notification events.
-

10.26 Future Enhancements

Future improvements may include:

- Apache Kafka Integration
 - RabbitMQ Message Broker
 - Workflow Designer UI
 - BPMN Workflow Engine
 - Multi-Channel Notification Routing
 - WhatsApp Notifications
 - Microsoft Teams Integration
 - Slack Integration
 - AI-Based Notification Personalization
 - Event Streaming Platform
-

10.27 Chapter Summary

This chapter defines the **Notification Architecture, Scheduler, Background Jobs, Automation, and Workflow Engine** for the Smart Education Platform. It establishes a scalable asynchronous processing model using Spring Scheduler, Spring Async, event-driven workflows, and a centralized notification engine. The architecture supports reliable communication across email, SMS, push, and in-app channels while ensuring high performance, fault tolerance, workflow automation, and enterprise-grade operational reliability.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 11 – Logging, Monitoring, Observability, Performance Engineering & Enterprise Operations

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Monitoring Stack: Prometheus + Grafana

Logging Stack: ELK Stack (Elasticsearch, Logstash, Kibana)

11.1 Introduction

An enterprise application must provide complete visibility into system behavior, application health, infrastructure performance, security events, and business operations.

The **Logging, Monitoring, Observability, Performance Engineering & Enterprise Operations** architecture enables proactive monitoring, rapid issue detection, performance optimization, troubleshooting, and operational excellence.

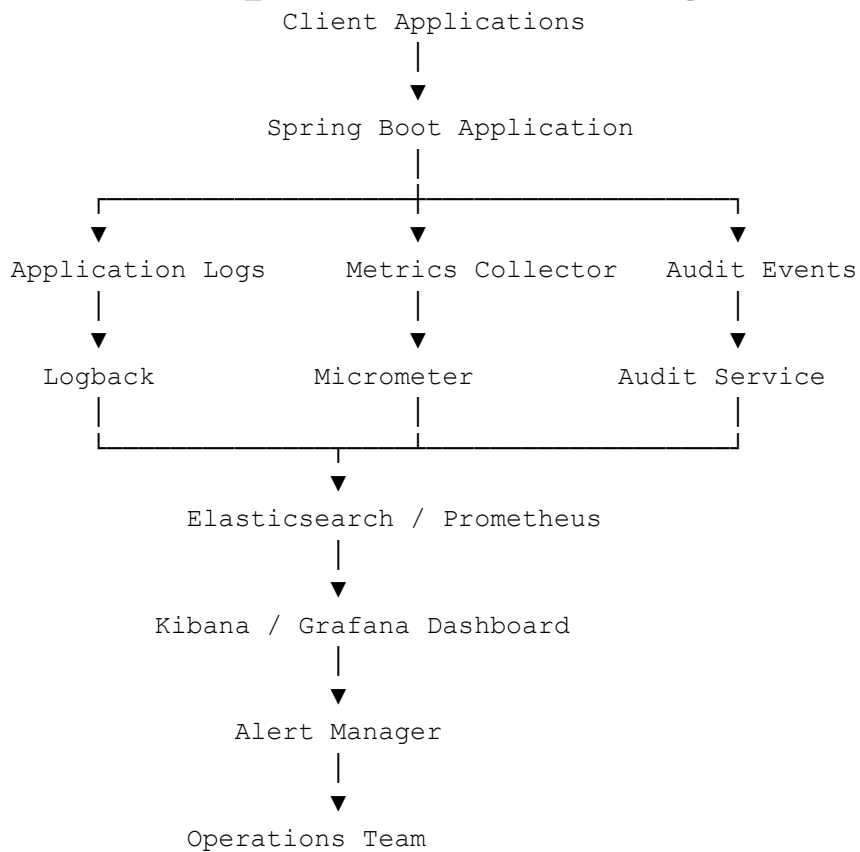
This architecture provides end-to-end visibility across the Smart Education Platform.

11.2 Objectives

The architecture is designed to provide:

- Centralized Logging
 - Real-Time Monitoring
 - Distributed Observability
 - Performance Analytics
 - Health Monitoring
 - Security Monitoring
 - Infrastructure Monitoring
 - Business Monitoring
 - Alert Management
 - Enterprise Operations
-

11.3 Enterprise Monitoring Architecture



11.4 Logging Architecture

The platform generates centralized logs for every important operation.

Log Categories

- Application Logs
 - API Logs
 - Security Logs
 - Audit Logs
 - Database Logs
 - Scheduler Logs
 - AI Logs
 - Notification Logs
 - Integration Logs
 - Performance Logs
-

11.5 Logging Flow

API Request

|

Controller

|

Business Service

|

Logger

|

Log Aggregator

|

Elasticsearch

|

Kibana Dashboard

11.6 Application Logging

Application logs capture:

- Startup Events
- Shutdown Events
- Business Operations
- Service Execution
- Cache Operations
- Database Calls
- Background Jobs
- Scheduler Execution

Recommended Log Levels

- TRACE

- DEBUG
 - INFO
 - WARN
 - ERROR
-

11.7 Security Logging

Record security-related activities:

- Login Success
 - Login Failure
 - Password Reset
 - Account Lock
 - Role Changes
 - Permission Changes
 - Token Refresh
 - Unauthorized Access
 - Suspicious Activity
-

11.8 Audit Logging

Audit every critical business event.

Examples

- User Registration
- Profile Update
- Resume Upload
- Resume Download
- Job Posting
- Job Application
- Interview Scheduling
- Offer Release
- AI Configuration Changes
- System Configuration Updates

Audit fields

- User ID

- Module
 - Action
 - Entity
 - Previous Value
 - New Value
 - IP Address
 - Device
 - Timestamp
-

11.9 API Logging

Capture information for every API request.

Fields

- Endpoint
 - HTTP Method
 - User ID
 - Request ID
 - Status Code
 - Response Time
 - Request Size
 - Response Size
 - Client IP
 - Timestamp
-

11.10 Error Logging

Log all exceptions.

Information stored

- Exception Name
- Stack Trace
- Module
- API
- User
- Request ID
- Severity

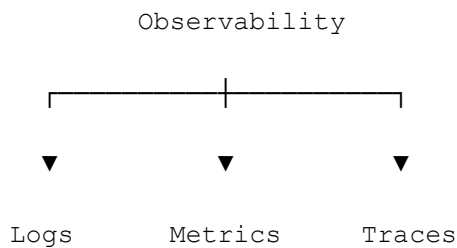
- Timestamp

Severity

- LOW
- MEDIUM
- HIGH
- CRITICAL

11.11 Observability Architecture

Observability consists of three pillars.



Together they provide complete system visibility.

11.12 Metrics Collection

Metrics are collected using **Micrometer** and exported to **Prometheus**.

Application Metrics

- API Count
- API Latency
- Request Rate
- Error Rate
- Success Rate
- Active Sessions
- Login Count

Business Metrics

- New Registrations
- Job Applications
- Assessment Attempts

- AI Requests
- Resume Downloads
- Placements
- Recruiter Activity

Infrastructure Metrics

- CPU
 - Memory
 - Disk
 - Network
 - JVM
 - Database
 - Redis
-

11.13 Distributed Tracing

Every request receives a unique **Trace ID**.

Client

↓

API Gateway

↓

Controller

↓

Service

↓

Repository

↓

Database

↓

Response

Trace IDs enable end-to-end request tracking across modules and future microservices.

11.14 Health Monitoring

Health checks include:

Application

- Running Status
- Memory Usage
- Thread Count

Database

- Connection Status
- Query Latency
- Active Connections

Redis

- Availability
- Memory Usage
- Cache Hit Ratio

Object Storage

- Connectivity
- Storage Capacity

AI Provider

- Availability
 - Response Time
-

11.15 Performance Engineering

Monitor

- API Response Time
- SQL Query Time
- Cache Hit Ratio

- JVM Performance
- Thread Pool Usage
- Garbage Collection
- Background Job Execution
- File Upload Time

Performance Targets

Metric	Target
Average API Response	<300 ms
Authentication	<150 ms
Dashboard Loading	<2 sec
Resume Generation	<5 sec
AI Response	<8 sec
File Upload	<5 sec
Cache Hit Ratio	>90%

11.16 Database Monitoring

Monitor continuously:

- Active Connections
 - Slow Queries
 - Deadlocks
 - Lock Wait Time
 - Transactions Per Second
 - Replication Status
 - Index Usage
 - Table Growth
-

11.17 Redis Monitoring

Track:

- Memory Usage
- Cache Hit Ratio
- Cache Miss Ratio
- Connected Clients

- Key Expiration
 - Evictions
 - Response Time
-

11.18 JVM Monitoring

Monitor:

- Heap Memory
 - Non-Heap Memory
 - Garbage Collection
 - Thread Count
 - CPU Usage
 - Class Loading
 - JVM Uptime
-

11.19 Dashboard Architecture

Grafana dashboards should include:

System Dashboard

- Server Health
- CPU
- Memory
- Disk
- Network

Application Dashboard

- API Response Time
- Active Users
- Error Rate
- Throughput

Database Dashboard

- Query Performance
- Connections

- Replication
- Storage

Business Dashboard

- Learner Registrations
 - Placements
 - Assessments
 - AI Usage
-

11.20 Alert Management

Alerts should be generated for:

Critical Alerts

- Application Down
- Database Down
- Redis Down
- Disk Full
- High CPU
- High Memory
- Authentication Failure Spike

Business Alerts

- AI Provider Failure
- Email Service Failure
- SMS Gateway Failure
- High Error Rate
- Slow APIs

Notification Channels

- Email
 - SMS
 - Slack (Future)
 - Microsoft Teams (Future)
-

11.21 Enterprise Operations

Daily Tasks

- Review Alerts
- Verify Backups
- Check API Performance
- Review Error Logs

Weekly Tasks

- Performance Analysis
- Capacity Review
- Security Audit
- Log Cleanup

Monthly Tasks

- Disaster Recovery Test
- Infrastructure Review
- Database Maintenance
- Performance Benchmarking

11.22 Log Retention Policy

Log Type	Retention
Application Logs	30 Days
API Logs	90 Days
Security Logs	180 Days
Audit Logs	7 Years (or organizational policy)
Error Logs	180 Days
Performance Metrics	1 Year

11.23 Performance Optimization Strategy

Optimization techniques:

- Redis Caching

- Connection Pooling
 - SQL Optimization
 - Batch Processing
 - Lazy Loading
 - Asynchronous Execution
 - Compression
 - CDN Integration
 - Materialized Views
-

11.24 Enterprise Best Practices

- Use structured JSON logging.
 - Include Correlation ID and Trace ID in every request.
 - Never log passwords, tokens, or sensitive personal data.
 - Monitor SLAs continuously.
 - Automate health checks.
 - Review slow queries regularly.
 - Archive old logs according to retention policies.
 - Test alerting rules periodically.
 - Separate operational dashboards from business dashboards.
-

11.25 Future Enhancements

Future improvements may include:

- OpenTelemetry Integration
 - Jaeger Distributed Tracing
 - Loki Log Aggregation
 - AI-Based Anomaly Detection
 - Predictive Infrastructure Monitoring
 - Self-Healing Operations
 - Automated Capacity Planning
 - Chaos Engineering
 - Service Mesh Observability
 - AIOps Platform Integration
-

11.26 Chapter Summary

This chapter defines the **Logging, Monitoring, Observability, Performance Engineering, and Enterprise Operations** architecture for the Smart Education Platform. It establishes a comprehensive operational framework using centralized logging, metrics collection, distributed tracing, health monitoring, performance engineering, and enterprise dashboards. Combined with proactive alerting and operational best practices, this architecture enables reliable, secure, high-performance production operations and continuous system improvement.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 11 – Logging, Monitoring, Observability, Performance Engineering & Enterprise Operations

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Logging Framework: Logback (SLF4J)

Metrics: Micrometer

Monitoring: Prometheus + Grafana

Log Analytics: ELK Stack (Elasticsearch, Logstash, Kibana)

Tracing: OpenTelemetry (Future Ready)

11.1 Introduction

Modern enterprise applications require complete visibility into application behavior, business transactions, infrastructure health, and security events.

The **Logging, Monitoring, Observability, Performance Engineering & Enterprise Operations Architecture** ensures that every component of the Smart Education Platform can be monitored, measured, analyzed, and optimized throughout its lifecycle.

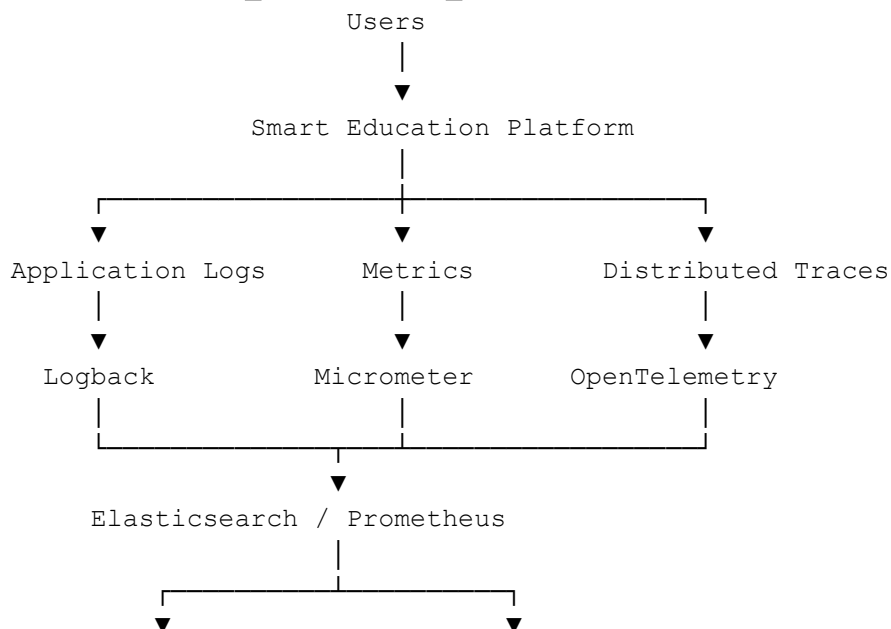
This architecture enables proactive issue detection, faster troubleshooting, operational intelligence, SLA monitoring, and continuous performance optimization.

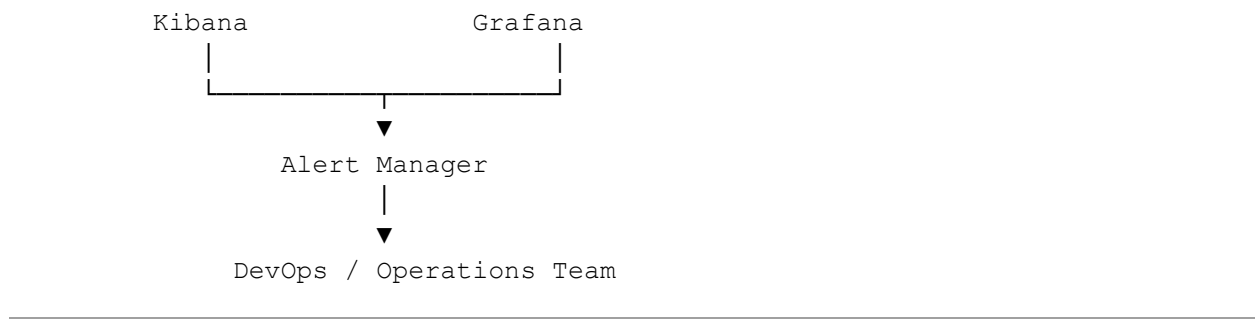
11.2 Architecture Objectives

The architecture provides:

- Centralized Logging
 - Real-Time Monitoring
 - Complete Observability
 - Distributed Tracing
 - Business Monitoring
 - Infrastructure Monitoring
 - Performance Engineering
 - Alert Management
 - Audit & Compliance
 - Production Operations
-

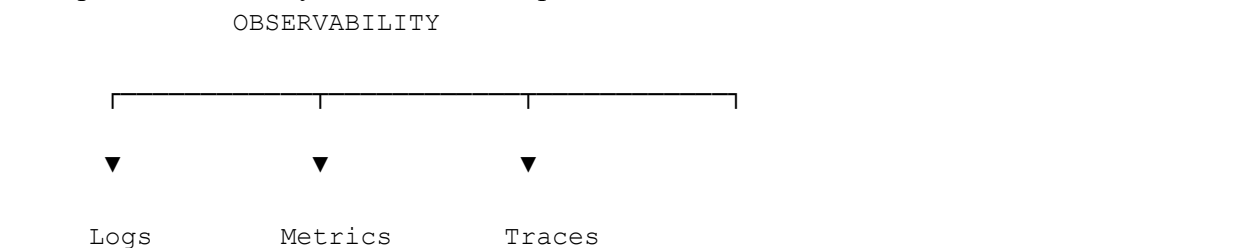
11.3 Enterprise Operations Architecture





11.4 Observability Model

Enterprise observability consists of three pillars.

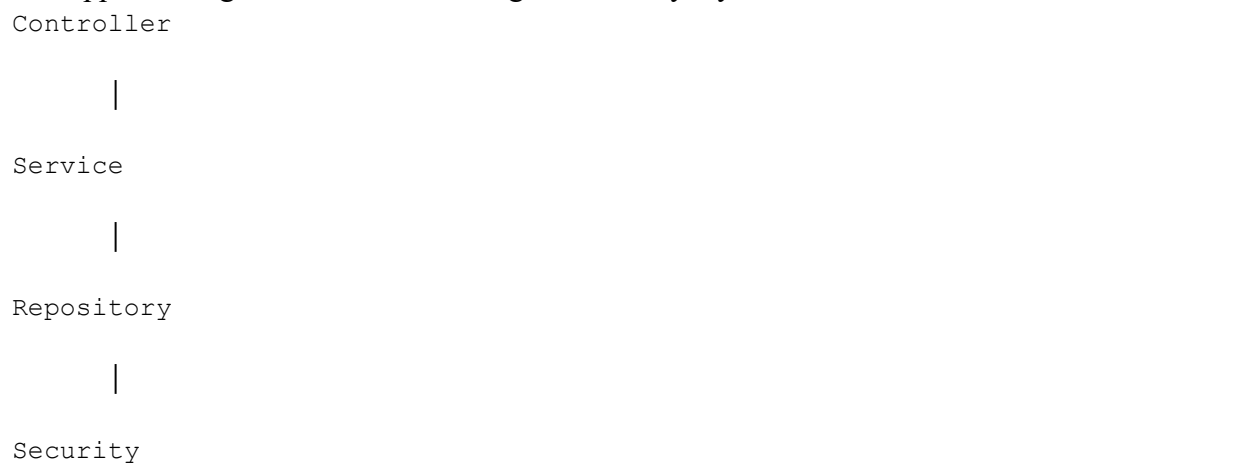


Each pillar complements the others.

Pillar	Purpose
Logs	What happened?
Metrics	How is the system performing?
Traces	Where did the request spend time?

11.5 Logging Architecture

The application generates structured logs from every layer.



|
Infrastructure

|
Logback

|
Logstash

|
Elasticsearch

|
Kibana

11.6 Logging Categories

The platform maintains multiple log streams.

Application Logs

- Startup
 - Shutdown
 - Business Operations
 - Service Execution
-

API Logs

- Endpoint
 - Request
 - Response
 - Response Time
 - HTTP Status
-

Security Logs

- Login
 - Logout
 - Password Reset
 - JWT Validation
 - Permission Denied
 - Token Refresh
 - Failed Authentication
-

Database Logs

- Slow Queries
 - Deadlocks
 - Transaction Failures
 - Connection Pool
-

Scheduler Logs

- Scheduled Jobs
 - Batch Jobs
 - Retry Jobs
 - Cleanup Jobs
-

AI Logs

- Prompt Execution
 - AI Provider
 - Token Usage
 - Response Time
 - Cost Tracking
-

Notification Logs

- Email
 - SMS
 - Push Notification
 - Delivery Status
-

Integration Logs

- Third-party APIs
 - Retry
 - Timeout
 - Circuit Breaker
-

Performance Logs

- Execution Time
 - Memory
 - CPU
 - Cache
-

11.7 Log Levels

Level	Usage
TRACE	Detailed debugging
DEBUG	Development debugging
INFO	Business events
WARN	Recoverable problems
ERROR	Failures
FATAL	Critical failures (if adopted by logging policy)

Production Recommendation

- INFO
 - WARN
 - ERROR
-

11.8 Structured Logging

Every log should contain:

- Timestamp
- Request ID
- Trace ID

- User ID
- Module
- API
- HTTP Method
- Execution Time
- Status
- Error Code
- Server Name

Example

Timestamp
TraceId
RequestId
UserId
Module
API
Duration
Status

11.9 Correlation ID

Every request receives a unique Correlation ID.

Client

↓

Correlation Filter

↓

Controller

↓

Service

↓

Repository

↓

Database

↓

Response

Benefits

- Easier debugging
- Complete request tracking
- Multi-service compatibility

11.10 Distributed Tracing

Future-ready architecture using OpenTelemetry.

API

↓

Authentication

↓

Business Service

↓

Database

↓

Redis

↓

External API

↓

Response

Each component contributes to one trace.

11.11 Metrics Collection

Micrometer collects application metrics.

Application Metrics

- API Count
 - Request Rate
 - Error Rate
 - Login Count
 - Active Users
 - Scheduler Jobs
-

JVM Metrics

- Heap Memory
 - Non Heap Memory
 - Thread Count
 - GC Time
 - CPU Usage
-

Database Metrics

- Connections
 - Slow Queries
 - Transactions
 - Lock Wait
 - Query Time
-

Redis Metrics

- Cache Hits
 - Cache Misses
 - Memory
 - Clients
 - Evictions
-

Business Metrics

- Registrations
- Job Applications

- Resume Downloads
 - AI Requests
 - Coding Submissions
 - SQL Attempts
 - Placements
-

11.12 Performance Engineering

Performance monitoring includes:

- API Latency
 - Database Latency
 - Cache Performance
 - JVM Performance
 - Thread Pools
 - Background Jobs
 - Network Calls
 - AI Response Time
-

11.13 Performance Targets

Metric	Target
Login	<150 ms
API Response	<300 ms
Dashboard	<2 sec
Resume Generation	<5 sec
AI Response	<8 sec
File Upload	<5 sec
Database Query	<100 ms
Cache Hit Ratio	>90%

11.14 Health Monitoring

Health endpoints monitor:

Application

- Running
- Startup
- Shutdown

Database

- Connectivity
- Replication
- Connections

Redis

- Memory
- Availability

Object Storage

- Bucket Access
- Capacity

SMTP

- Connectivity

AI Provider

- Availability
- Response Time

11.15 Dashboard Architecture

Grafana dashboards.

Executive Dashboard

- Active Users
- Placements
- Revenue
- AI Usage

DevOps Dashboard

- CPU
 - Memory
 - Disk
 - Containers
 - Network
-

Application Dashboard

- APIs
 - Errors
 - Response Time
 - Active Sessions
-

Database Dashboard

- Queries
 - Locks
 - Connections
 - Storage
-

AI Dashboard

- Requests
 - Tokens
 - Cost
 - Latency
-

11.16 Alert Management

Automatic alerts for:

Infrastructure

- CPU >80%
- Memory >85%
- Disk >90%
- Database Down
- Redis Down

Application

- API Failure
- High Error Rate
- High Latency
- Thread Exhaustion

Business

- AI Failure
 - Email Failure
 - SMS Failure
 - Payment Failure
 - Authentication Spike
-

11.17 Enterprise Operations

Daily Activities

- Review Logs
- Verify Backups
- Check Health Dashboard
- Review Alerts

Weekly Activities

- Performance Review
- Capacity Analysis
- Database Maintenance
- Security Audit

Monthly Activities

- Disaster Recovery Testing
- Infrastructure Upgrade Review
- Dependency Updates
- SLA Review

Quarterly Activities

- Load Testing
- Security Penetration Testing

- Performance Benchmarking
 - Capacity Planning
-

11.18 Log Retention Policy

Log Type	Retention
Application Logs	30 Days
API Logs	90 Days
Error Logs	180 Days
Security Logs	180 Days
Audit Logs	7 Years (or organizational policy)
Performance Metrics	1 Year

11.19 Performance Optimization Strategy

Techniques include:

- Redis Caching
 - Database Indexing
 - Materialized Views
 - Connection Pooling
 - Batch Processing
 - Lazy Loading
 - Async Processing
 - Compression
 - CDN
 - Pagination
-

11.20 Capacity Planning

Monitor growth of:

- Users
- Database Size
- Redis Memory
- Uploaded Files

- Audit Logs
- AI Usage
- API Traffic

Capacity should be reviewed regularly to ensure sufficient compute, storage, and network resources.

11.21 Security Monitoring

Monitor continuously:

- Failed Logins
- Unauthorized API Access
- Privilege Escalation
- Token Abuse
- Suspicious Downloads
- SQL Injection Attempts
- XSS Attempts
- Brute Force Attacks

Critical security events should generate immediate alerts.

11.22 Enterprise Best Practices

- Use structured JSON logging.
 - Include Correlation ID and Trace ID in every request.
 - Never log passwords, JWTs, OTPs, or sensitive personal data.
 - Centralize logs in Elasticsearch.
 - Monitor SLAs continuously.
 - Automate health checks and alerting.
 - Review slow queries regularly.
 - Archive logs according to retention policies.
 - Validate backup and restore procedures periodically.
 - Document operational runbooks for incident response.
-

11.23 Future Enhancements

Future releases may include:

- OpenTelemetry Full Integration
 - Jaeger Distributed Tracing
 - Loki Log Aggregation
 - AI-Based Anomaly Detection
 - Predictive Infrastructure Monitoring
 - Self-Healing Services
 - AIOps Platform
 - Automated Incident Response
 - Chaos Engineering
 - Real-Time Business Intelligence Dashboards
-

11.24 Chapter Summary

This chapter defines the **Logging, Monitoring, Observability, Performance Engineering, and Enterprise Operations** architecture for the Smart Education Platform. It provides a comprehensive operational framework using centralized logging, metrics collection, distributed tracing, health monitoring, alert management, and performance engineering. By combining observability with structured operational practices, the platform can achieve high reliability, rapid issue resolution, proactive monitoring, and enterprise-grade production operations.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 12 – Scalability, High Availability, Cloud Architecture, DevOps, CI/CD & Production Deployment

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Deployment Platform: Docker, Kubernetes Ready

CI/CD Platform: GitHub Actions

12.1 Introduction

The **Scalability, High Availability, Cloud Architecture, DevOps, CI/CD, and Production Deployment** architecture ensures that the Smart Education Platform can reliably support increasing user traffic, maintain high availability, automate deployments, and recover quickly from failures.

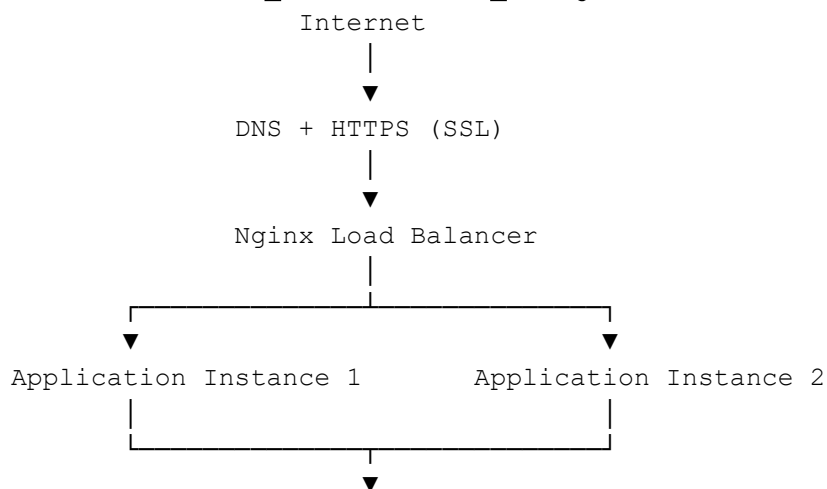
The architecture is designed with cloud-native principles, enabling seamless scaling, automated deployments, continuous monitoring, and zero-downtime releases.

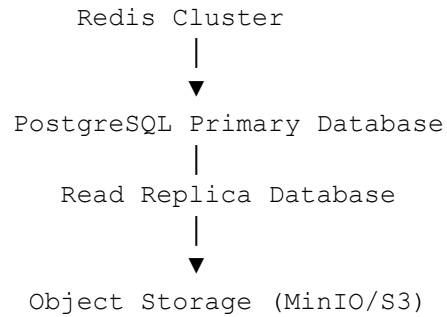
12.2 Objectives

The architecture is designed to provide:

- Horizontal Scalability
 - High Availability
 - Cloud-Native Deployment
 - Automated CI/CD
 - Zero-Downtime Releases
 - Disaster Recovery
 - Infrastructure Automation
 - Secure Deployments
 - Production Monitoring
 - Enterprise Reliability
-

12.3 Enterprise Deployment Architecture





12.4 Scalability Strategy

The platform supports:

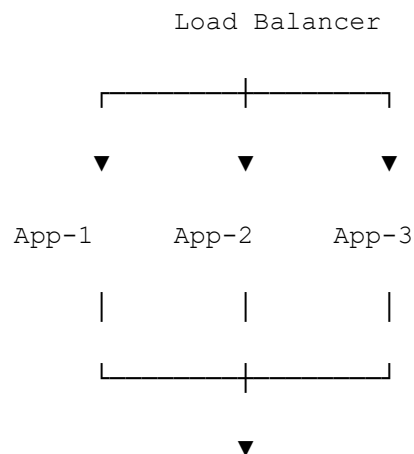
Horizontal Scaling

- Multiple Application Instances
- Stateless REST APIs
- Shared Redis Cache
- Load Balancer Distribution

Vertical Scaling

- CPU Upgrade
- Memory Upgrade
- Storage Expansion
- Database Resource Scaling

12.5 Horizontal Scaling



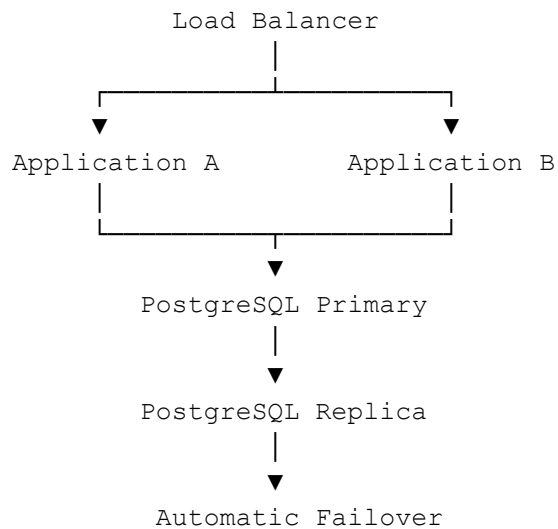
Shared Database + Redis

Benefits:

- Higher throughput
 - Fault tolerance
 - Easy expansion
 - Improved availability
-

12.6 High Availability Architecture

The application is designed with redundant components.



High Availability Features

- Multiple application nodes
 - Database replication
 - Redis persistence
 - Health monitoring
 - Automatic failover
-

12.7 Cloud Architecture

Supported cloud platforms:

- AWS

- Microsoft Azure
- Google Cloud Platform (GCP)
- Private Cloud
- On-Premises Infrastructure

Cloud Components

- Virtual Machines
 - Load Balancers
 - Managed PostgreSQL
 - Managed Redis
 - Object Storage
 - Monitoring Services
-

12.8 Container Architecture

The application is containerized using Docker.

React Application

↓

Docker Image

↓

Container

↓

Production Server

Spring Boot Application

↓

Docker Image

↓

Container

↓

Production Server

Benefits

- Consistent deployments
- Environment isolation
- Easy rollback
- Rapid scaling

12.9 Kubernetes Readiness

Future deployment architecture:

Kubernetes Cluster

|

Deployment

|

Replica Set

|

Pods

|

Services

|

Ingress

Features

- Auto Scaling
- Rolling Updates
- Self-Healing
- Load Distribution

12.10 CI/CD Pipeline

Developer

↓
GitHub
↓
Pull Request
↓
Code Review
↓
GitHub Actions
↓
Build
↓
Unit Tests
↓
Integration Tests
↓
Docker Image
↓
Deploy QA
↓
Deploy UAT
↓
Deploy Production

12.11 Continuous Integration

CI automatically performs:

- Code Compilation
 - Static Code Analysis
 - Unit Testing
 - Dependency Checks
 - Security Scanning
 - Build Verification
 - Artifact Generation
-

12.12 Continuous Deployment

Deployment pipeline:

Development

↓

QA

↓

UAT

↓

Production

Each stage requires:

- Build Validation
 - Automated Tests
 - Health Checks
 - Deployment Approval (for production)
-

12.13 Deployment Strategies

Supported deployment models:

Rolling Deployment

Gradually replaces old instances.

Blue-Green Deployment

Current Version

↓

Blue Environment

↓

Green Environment

↓

Traffic Switch

Canary Deployment (Future)

Release to a small percentage of users before full rollout.

12.14 Infrastructure as Code (IaC)

Recommended tools:

- Terraform
- Ansible
- Kubernetes YAML
- Docker Compose

Benefits

- Repeatable deployments
 - Version-controlled infrastructure
 - Automated provisioning
 - Reduced configuration drift
-

12.15 Production Release Workflow

Feature Development

↓

Code Review

↓

Merge

↓

CI Build

↓

QA Testing

↓

UAT Approval

↓

Production Release

↓

Monitoring

↓

Rollback (If Required)

12.16 Environment Architecture

Environment	Purpose
Local	Developer machine
Development	Feature integration
QA	Functional testing
UAT	Business validation
Staging	Production-like testing
Production	Live users

Each environment has separate configuration, databases, and secrets.

12.17 Configuration Management

Configuration includes:

- Database
- Redis
- JWT
- SMTP
- SMS Gateway
- AI Provider
- File Storage
- Logging
- Monitoring

Best Practices

- Environment variables
 - Secret managers
 - Externalized configuration
 - Profile-based settings
-

12.18 Disaster Recovery

Recovery Components

- Database Backups
- Redis Persistence
- Object Storage Replication
- Infrastructure Backup
- Configuration Backup

Targets

Metric

Target

Recovery Point Objective (RPO) ≤ 15 Minutes

Recovery Time Objective (RTO) ≤ 1 Hour

12.19 Performance Optimization

Production optimizations include:

- Redis Caching
 - HikariCP Connection Pooling
 - Database Indexing
 - Compression
 - HTTP Keep-Alive
 - GZIP Compression
 - CDN Integration
 - Lazy Loading
 - Batch Processing
-

12.20 Security in DevOps

Security controls include:

- HTTPS Everywhere
 - Container Image Scanning
 - Secret Management
 - Dependency Vulnerability Scanning
 - Least Privilege Access
 - Secure Build Pipelines
 - Automated Security Testing
 - Audit Logging
-

12.21 Production Monitoring

Monitor continuously:

Application

- Response Time
- Error Rate
- Active Sessions

Infrastructure

- CPU
- Memory

- Disk
- Network

Database

- Connections
- Slow Queries
- Replication

Redis

- Cache Hit Ratio
- Memory Usage

Business

- Registrations
- AI Usage
- Placements
- Notifications

12.22 Rollback Strategy

Rollback triggers:

- Failed Deployment
- High Error Rate
- Critical Bug
- Performance Degradation
- Security Issue

Rollback Process

Deploy

↓

Health Check

↓

Failure?

↓

Rollback

↓

Previous Stable Version

12.23 Production Readiness Checklist

Before production deployment:

- All tests passed
 - Code review completed
 - Security scan completed
 - Performance testing completed
 - Backup verified
 - Monitoring configured
 - Alerts configured
 - SSL certificates installed
 - Database migrations verified
 - Rollback plan documented
-

12.24 Enterprise DevOps Best Practices

- Use immutable Docker images.
 - Automate the entire CI/CD pipeline.
 - Keep infrastructure under version control.
 - Separate application configuration from code.
 - Monitor deployments in real time.
 - Perform regular backup and recovery testing.
 - Apply least-privilege access for deployment pipelines.
 - Conduct post-release reviews for continuous improvement.
-

12.25 Future Enhancements

Future improvements may include:

- Kubernetes Production Deployment

- Service Mesh (Istio)
 - GitOps with Argo CD
 - Multi-Region Active-Active Deployment
 - Auto Scaling Based on Load
 - Chaos Engineering
 - Progressive Delivery
 - Serverless Background Workers
 - Edge Computing
 - AI-Assisted Infrastructure Optimization
-

12.26 Chapter Summary

This chapter defines the **Scalability, High Availability, Cloud Architecture, DevOps, CI/CD, and Production Deployment** strategy for the Smart Education Platform. It outlines how the platform achieves enterprise-grade scalability through stateless application design, load balancing, containerization, automated CI/CD pipelines, high availability, disaster recovery, and cloud-ready infrastructure. These practices ensure reliable, secure, and efficient deployments while enabling the platform to scale seamlessly as user demand grows.

Software Architecture Document (SAD)

Smart Education Platform (Phase 1)

Chapter 13 – Architecture Governance, Coding Standards, Quality Assurance, Technical Guidelines & Future Roadmap

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith (Microservice Ready)

Technology Stack: Java 21, Spring Boot 3.x, React 19, PostgreSQL, Redis, Docker

13.1 Introduction

A successful enterprise software product depends not only on its architecture but also on consistent engineering practices, governance, coding standards, quality assurance processes, and long-term technical planning.

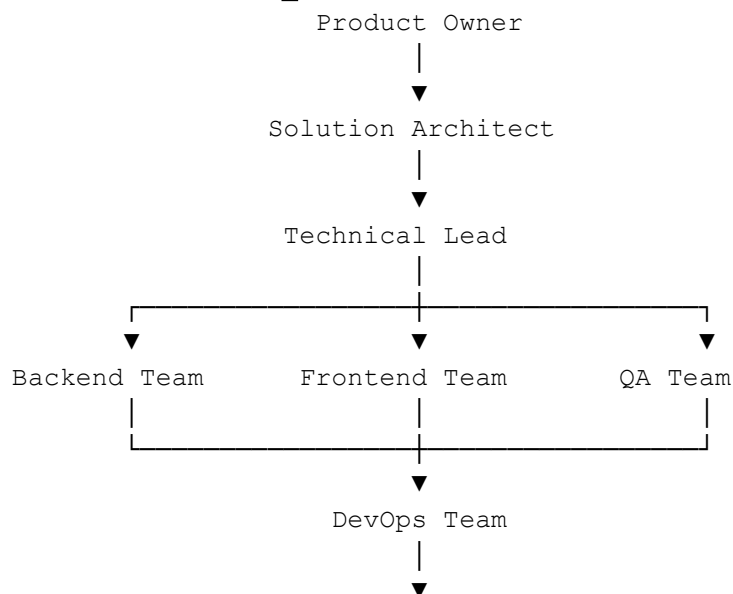
This chapter establishes the engineering governance model for the Smart Education Platform to ensure that development remains scalable, maintainable, secure, and aligned with business objectives throughout the project lifecycle.

13.2 Objectives

The architecture governance framework aims to provide:

- Standardized Development Practices
 - Consistent Code Quality
 - Architecture Compliance
 - Secure Software Development
 - Enterprise Coding Standards
 - Continuous Quality Assurance
 - Technical Documentation Standards
 - Release Governance
 - Future Scalability
 - Long-Term Maintainability
-

13.3 Enterprise Governance Model



13.4 Architecture Governance Principles

The platform follows these principles:

- Clean Architecture
 - SOLID Principles
 - Domain-Driven Design (DDD)
 - API-First Development
 - Security by Design
 - Cloud-Native Readiness
 - Performance by Design
 - Test-Driven Development (where applicable)
 - Continuous Improvement
-

13.5 Development Standards

Every feature must follow a standard development lifecycle.

Requirement

↓

Architecture Review

↓

Technical Design

↓

Development

↓

Code Review

↓

Unit Testing

↓

Integration Testing

↓

QA Validation

↓

UAT

↓

Production Release

13.6 Coding Standards

Java Standards

- Java 21 LTS
 - Spring Boot 3.x
 - Constructor Injection
 - Immutable DTOs where appropriate
 - Use Optional appropriately
 - Avoid NullPointerException
 - Proper exception hierarchy
 - Use Streams judiciously
 - Prefer interfaces over implementations
-

Spring Boot Standards

- One responsibility per service
 - Thin controllers
 - Business logic only in service layer
 - Repository only for persistence
 - DTO-based API communication
 - Global exception handling
 - Centralized validation
 - Standard API responses
-

React Standards

- Functional Components
 - React Hooks
 - Component Reusability
 - State Management using Redux Toolkit
 - React Query for server state
 - Lazy Loading
 - Code Splitting
 - Reusable UI Components
-

13.7 Naming Conventions

Packages

`com.smarteducation.module`

Examples

`com.smarteducation.authentication`

`com.smarteducation.learner`

`com.smarteducation.resume`

Classes

Examples

`UserService`

`ResumeController`

`AssessmentRepository`

`JobApplicationDTO`

Methods

Examples

`createUser()`

`updateResume()`

```
calculateScore()
```

```
generateReport()
```

Variables

```
userName
```

```
totalScore
```

```
interviewDate
```

```
placementStatus
```

Constants

```
MAX_LOGIN_ATTEMPTS
```

```
DEFAULT_PAGE_SIZE
```

```
JWT_EXPIRATION_TIME
```

13.8 Code Organization Standards

Each module should follow:

```
module
```

```
|— controller
```

```
|— service
```

```
|   └─ impl
```

```
|— repository
```

```
|— entity
```

```
|— dto
```

```
|— mapper
```

```
|— validator
```

```
|— exception
```

|— util

|— config

13.9 Code Review Guidelines

Every Pull Request should verify:

- Coding Standards
 - Business Logic
 - Security
 - Performance
 - Exception Handling
 - Logging
 - Test Coverage
 - Documentation
 - Naming Standards
 - Design Principles
-

13.10 Quality Assurance Framework

Testing Levels

Unit Testing

↓

Integration Testing

↓

API Testing

↓

UI Testing

↓

System Testing

↓

Performance Testing

↓

Security Testing

↓

UAT

13.11 Quality Gates

A feature can be merged only if:

- Code Review Approved
 - Build Successful
 - Unit Tests Passed
 - Integration Tests Passed
 - Static Analysis Passed
 - Security Scan Passed
 - No Critical Bugs
 - Documentation Updated
-

13.12 Static Code Analysis

Recommended tools

Backend

- SonarQube
- SpotBugs
- Checkstyle
- PMD

Frontend

- ESLint
- Prettier

Benefits

- Better code quality
 - Maintainability
 - Reduced technical debt
-

13.13 Documentation Standards

Maintain documentation for:

- API Documentation
- Database Changes
- Architecture Decisions
- Deployment Guides
- User Manuals
- Release Notes
- Configuration Guides

Every major feature should include technical documentation.

13.14 Version Control Strategy

Recommended Git Workflow

main

|

develop

|

feature/*

|

release/*

|

hotfix/*

Commit Message Examples

feat: Add AI resume review

fix: Resolve JWT refresh issue

refactor: Optimize placement workflow

docs: Update API documentation

13.15 API Governance

API standards

- RESTful Design
 - Versioning (/api/v1)
 - Standard Response Format
 - Consistent Error Responses
 - Pagination Support
 - Filtering
 - Sorting
 - OpenAPI Documentation
-

13.16 Security Guidelines

Developers must:

- Validate all inputs
 - Never hardcode credentials
 - Use parameterized queries
 - Hash passwords with BCrypt
 - Protect JWT secrets
 - Encrypt sensitive information
 - Follow OWASP recommendations
 - Log security events
-

13.17 Performance Guidelines

Recommendations

- Cache frequently used data
- Use pagination

- Optimize SQL queries
 - Avoid N+1 queries
 - Batch updates
 - Compress responses
 - Lazy loading where appropriate
 - Monitor slow APIs
-

13.18 Technical Debt Management

Technical debt should be:

- Identified
- Prioritized
- Documented
- Reviewed
- Scheduled for resolution

Categories

- Code Smells
 - Duplicate Code
 - Legacy APIs
 - Performance Issues
 - Security Improvements
-

13.19 Architecture Decision Records (ADR)

Major architectural decisions should be documented.

Each ADR should include:

- Decision Title
- Context
- Problem Statement
- Options Considered
- Selected Solution
- Consequences
- Date
- Decision Owner

Examples

- Why PostgreSQL?
 - Why Spring Boot?
 - Why React?
 - Why Redis?
 - Why JWT?
 - Why Modular Monolith?
-

13.20 Release Governance

Production releases should include:

- Code Freeze
 - Release Candidate
 - QA Approval
 - UAT Approval
 - Database Migration Validation
 - Deployment Checklist
 - Rollback Plan
 - Production Monitoring
-

13.21 Engineering Best Practices

Development teams should:

- Follow SOLID principles.
 - Keep methods small and focused.
 - Write meaningful commit messages.
 - Review all pull requests.
 - Maintain high test coverage.
 - Refactor continuously.
 - Document architectural decisions.
 - Keep dependencies up to date.
 - Automate repetitive tasks.
-

13.22 Future Architecture Roadmap

Phase 2

- Microservices Migration
- Event Streaming (Kafka)
- Kubernetes Production
- Multi-Tenant Support
- AI Agents
- Vector Database Integration

Phase 3

- Global Multi-Region Deployment
 - Real-Time Collaboration
 - AI Personalization Engine
 - Service Mesh
 - CQRS & Event Sourcing
 - Predictive Analytics
-

13.23 Enterprise Development Checklist

Before every release:

- Requirements Approved
 - Architecture Reviewed
 - Coding Standards Followed
 - Unit Tests Completed
 - Integration Tests Completed
 - API Documentation Updated
 - Security Review Completed
 - Performance Testing Completed
 - Monitoring Configured
 - Rollback Plan Ready
-

13.24 Success Metrics

Engineering success will be measured through:

Metric	Target
Unit Test Coverage	≥ 80%

Metric	Target
Build Success Rate	≥ 99%
Code Review Compliance	100%
Critical Security Vulnerabilities	0
Production Deployment Success	≥ 99%
Mean Time to Recovery (MTTR)	< 1 Hour
Production Uptime	≥ 99.9%
API Availability	≥ 99.9%

13.25 Chapter Summary

This chapter establishes the **Architecture Governance, Coding Standards, Quality Assurance Framework, Technical Guidelines, and Future Roadmap** for the Smart Education Platform. It defines enterprise engineering practices that ensure consistent code quality, secure development, standardized architecture, effective release governance, and continuous improvement. By following these guidelines, development teams can deliver scalable, maintainable, secure, and production-ready software while supporting future architectural evolution.